

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК**

Вольхин Данил Федорович

**МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ
ОРГАНИЗАЦИЯ КООПЕРАТИВНОГО ВОСПРИЯТИЯ СЦЕНЫ В ГРУППЕ
АВТОНОМНЫХ АГЕНТОВ НА ОСНОВЕ ОБМЕНА ВИЗУАЛЬНОЙ
ИНФОРМАЦИЕЙ
COOPERATIVE VISUAL PERCEPTION IN MULTI-AGENT SYSTEMS**

по направлению подготовки 01.04.02 Прикладная математика и информатика
образовательная программа «Исследование и предпринимательство в
искусственном интеллекте»

Научный руководитель

д-р наук/проф/должность

_____И.С. Копылов_____

Студент

_____Д.Ф. Вольхин_____

АННОТАЦИЯ

Работа посвящена проектированию и реализации многоагентной робототехнической платформы AgentsSwarm, обеспечивающей кооперативное восприятие сцены группой мобильных роботов, стандартизованное управление флотом и высокоуровневую оркестрацию задач посредством больших языковых моделей.

Цель работы — создание интегрированного программного комплекса, в котором запрос пользователя на естественном языке транслируется в скоординированное выполнение миссий группой роботов через единый сквозной контур управления.

Задачи: анализ подходов к кооперативному восприятию и координации многороботных систем; разработка микросервисной архитектуры из восьми взаимосвязанных компонентов; реализация LLM-оркестратора на базе OpenAI Agents SDK с девятью специализированными агентами (Router, MissionPlanner, MapAnalyst, Navigation, Charging, Patrol, Inspection, FleetOps, SwarmCoordinator); глубокая доработка трёх MCP-серверов (Mission Control, Mission Dispatch, ROS MSP) с переходом на асинхронную модель вызовов; адаптация форков NVIDIA Mission Control и Mission Dispatch под протокол VDA5050; экспериментальная оптимизация VLA-модели SmolVLA для бортового инференса.

Методология основана на принципах облачной робототехники, событийно-ориентированной архитектуры и протокола Model Context Protocol. Эмпирическая база включает симуляцию флота роботов Carter в NVIDIA Isaac Sim (headless-режим, ROS 2 Jazzy, rosbridge WebSocket); развёртывание LLM-сервиса Qwen2.5-Instruct в режиме Data Parallel на двух узлах Tesla V100 (32 ГБ VRAM) через авторский vLLM-сервис; пайплайн дистилляции, прунинга и квантизации модели SmolVLA.

Результаты: реализованы восемь самостоятельных компонентов — Interface (React + FastAPI + Celery, 168 коммитов, v0.5.0), Orchestrator (FastAPI + Agents SDK, 56 коммитов, v0.5.6), vLLM Service (Data Parallel, v0.2.9), доработанные форки Mission Control и Mission Dispatch, настроенный ROS 2 workspace, три переработанных MCP-сервера. Пройдено 43 из 43 интеграционных тестов API оркестратора. Для модели SmolVLA достигнуто сжатие в 443 раза по числу параметров при сохранении более 90% точности

(MSE +9,1%, R^2 -1,8%) и ускорении инференса в 25 раз (450 мс $\rightarrow$ 18 мс, RTX 4090).

ABSTRACT

This thesis presents the design and implementation of AgentsSwarm, a multi-agent robotic platform that enables cooperative scene perception among a group of mobile robots, standardised fleet management, and high-level task orchestration driven by large language models.

The objective is to build an integrated software system in which a natural-language user request is translated into coordinated multi-robot mission execution through a unified end-to-end control pipeline.

Research tasks: analysis of cooperative perception and multi-robot coordination approaches; design of a microservice architecture comprising eight interconnected components; development of an LLM orchestrator based on the OpenAI Agents SDK with nine specialised agents (Router, MissionPlanner, MapAnalyst, Navigation, Charging, Patrol, Inspection, FleetOps, SwarmCoordinator); deep rework of three MCP servers (Mission Control, Mission Dispatch, ROS MSP) with a full migration to asynchronous tool calls; adaptation of NVIDIA Mission Control and Mission Dispatch forks to the VDA5050 protocol; and experimental optimisation of the SmolVLA vision-language-action model for on-board inference.

The methodology is grounded in cloud-robotics principles, event-driven architecture, and the Model Context Protocol. The empirical base comprises simulation of a Carter robot fleet in NVIDIA Isaac Sim (headless mode, ROS 2 Jazzy, rosbridge WebSocket); deployment of a Qwen2.5-Instruct LLM service in Data Parallel mode across two Tesla V100 nodes (32 GB VRAM each) via a custom-built vLLM service; and a knowledge-distillation, pruning, and quantisation pipeline for SmolVLA.

Results: eight independent components were fully implemented — Interface (React + FastAPI + Celery, 168 commits, v0.5.0), Orchestrator (FastAPI + Agents SDK, 56 commits, v0.5.6), vLLM Service (Data Parallel, v0.2.9), reworked Mission Control and Mission Dispatch forks, a configured ROS 2 workspace, and three heavily modified MCP servers. All 43 orchestrator API integration tests passed. The SmolVLA model was compressed 443-fold in parameter count while retaining over 90% accuracy (MSE +9.1%, R^2 -1.8%) with a $25\times$ inference speed-up (450 ms $\rightarrow$ 18 ms on RTX 4090).

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 Анализ предметной области и обзор существующих решений	11
1.1 Кооперативное визуальное восприятие в многоагентных системах	11
1.1.1 Постановка задачи и фундаментальный компромисс	11
1.1.2 Ключевые алгоритмические подходы	11
1.2 Совместное картографирование и локализация	12
1.3 Координация флота и стандарты интероперабельности	13
1.3.1 VDA5050 как промышленный стандарт	13
1.3.2 OpenRMF и архитектурные аналоги	13
1.3.3 Behavior Trees как исполнимый формат миссий	13
1.3.4 Облачная и туманная робототехника	14
1.4 Большие языковые модели в робототехнике	14
1.4.1 Смена парадигмы: от чат-бота к reasoning-агенту	14
1.4.2 Парадигма ReAct и tool use	14
1.4.3 Безопасность и формальные ограничения LLM-планирования	15
1.4.4 LLM для координации роя	15
1.5 Foundation Models и Vision-Language-Action модели	15
1.6 Симуляционные среды и цифровые двойники	16
1.7 Выводы и результаты по главе 1	16
2 Проектирование архитектуры системы AgentsSwarm	18
2.1 Требования к системе	18
2.2 Общая микросервисная архитектура	19
2.3 Протокол взаимодействия агентов: Model Context Protocol	20
2.4 Протокол управления флотом: VDA5050	22
2.5 Агентная архитектура оркестратора	22

2.6	Событийно-ориентированная потоковая модель	23
2.7	Инфраструктура развёртывания	24
2.8	Выводы и результаты по главе 2	25
3	Реализация компонентов системы	26
3.1	Симуляционная среда: NVIDIA Isaac Sim и ROS 2 Workspace .	26
3.2	Mission Control: форк NVIDIA и авторские доработки	27
3.3	Mission Dispatch: форк NVIDIA и авторские доработки	28
3.4	MCP-серверы: глубокая переработка официальных реализаций	28
3.5	vLLM Service: авторский сервис инференса	29
3.6	Orchestrator: главный интеллектуальный микросервис	30
3.7	Interface: веб-платформа управления	32
3.7.1	Общая архитектура и стек	32
3.7.2	Backend: агенты и обработка задач	35
3.7.3	Потоковая модель: Redis Streams и WebSocket	35
3.7.4	Frontend: страницы интерфейса	36
3.7.5	Демонстрация рабочего воркфлоу	36
3.8	Выводы и результаты по главе 3	37
4	Оптимизация модели SmolVLA для бортового инференса	38
4.1	Постановка задачи и мотивация	38
4.2	Архитектура SmolVLA и исходные характеристики	38
4.3	Пайплайн оптимизации	39
4.4	Результаты экспериментов	40
4.5	Выводы и результаты по главе 4	40
5	Верификация системы	42
5.1	Стратегия тестирования	42
5.2	Тестирование оркестратора	42
5.3	Тестирование Interface	42
5.4	Выводы и результаты по главе 5	43
	ЗАКЛЮЧЕНИЕ	44
	БИБЛИОГРАФИЧЕСКИЙ СПИСОК	45
	ПРИЛОЖЕНИЯ	52
A	Глоссарий предметной области	53
	Приложение А. Глоссарий предметной области	53

Б	Список сокращений	55
	Приложение Б. Список сокращений	55
В	Технические характеристики инфраструктуры	56
	Приложение В. Технические характеристики инфраструктуры	56
Г	Ключевые фрагменты кода	57
	Приложение Г. Ключевые фрагменты кода	57
Д	Протоколы экспериментов	59
	Приложение Д. Протоколы экспериментов	59

ВВЕДЕНИЕ

Современная промышленная робототехника переживает качественный переход от использования одиночных роботов-исполнителей к развёртыванию гетерогенных флотов автономных мобильных роботов (AMR) и автоматизированных транспортных средств (AGV). Задачи складской логистики, промышленной инспекции, гибкого производства всё чаще требуют координированного взаимодействия группы роботов, способных воспринимать и интерпретировать общее пространство операций. Одиночный агент принципиально ограничен угловым полем зрения, дальностью сенсоров и локальными вычислительными ресурсами; группа же агентов, обменивающихся перцептивной информацией, способна совместно формировать наблюдение сцены, недоступное ни одному из них в отдельности.

Вместе с тем координация флота остаётся нетривиальной инженерной задачей. Существующие fleet management системы — в том числе соответствующие стандарту VDA5050 — предоставляют надёжный низкоуровневый контур диспетчеризации миссий, однако не обеспечивают высокоуровневого семантического планирования и не предоставляют пользователю интерфейс на естественном языке. Академические прототипы кооперативного восприятия (V2VNet, Where2comm, CoBEVT) достигают высоких показателей точности на симулированных датасетах, однако не интегрированы с промышленными стандартами координации флота.

Появление мощных больших языковых моделей (LLM) открывает принципиально новую возможность: транслировать пользовательский запрос на естественном языке напрямую в координированное выполнение многороботных миссий. Парадигмы ReAct и tool use превращают языковую модель из «чат-бота» в полноценный reasoning-агент, способный вызывать внешние инструменты, накапливать наблюдения и динамически корректировать план действий.

Объект исследования — многоагентная робототехническая система с кооперативным восприятием и централизованным fleet management.

Предмет исследования — методы организации взаимодействия, координации и LLM-планирования в группе автономных агентов.

Методы исследования: системный анализ, микросервисное проектирование, симуляционный эксперимент, дистилляция нейронных сетей.

Цель работы — создание интегрированного программного комплекса AgentsSwarm, в котором запрос пользователя на естественном языке транслируется в скоординированное выполнение миссий группой роботов через единый сквозной контур управления.

Для достижения цели решены следующие **задачи**:

1. Анализ подходов к кооперативному визуальному восприятию, совместному картографированию, координации флота и применению LLM в робототехнике.
2. Проектирование микросервисной архитектуры из восьми взаимосвязанных компонентов.
3. Реализация LLM-оркестратора на базе OpenAI Agents SDK с девятью специализированными агентами.
4. Глубокая переработка трёх MCP-серверов с переходом на асинхронную модель вызовов.
5. Адаптация форков NVIDIA Mission Control и Mission Dispatch под протокол VDA5050.
6. Настройка симуляционной среды NVIDIA Isaac Sim с группой роботов Carter.
7. Развёртывание vLLM-сервиса в режиме Data Parallel на кластере из двух узлов Tesla V100.
8. Экспериментальная оптимизация VLA-модели SmolVLA для бортового инференса.

Основные результаты. Реализованы восемь самостоятельных компонентов: Interface (v0.5.0, 168 коммитов), Orchestrator (v0.5.6, 56 коммитов), vLLM Service (v0.2.9), доработанные форки Mission Control и Mission Dispatch, настроенный ROS 2 workspace, три переработанных MCP-сервера. Пройдено 43 из 43 интеграционных тестов. Для SmolVLA достигнуто сжатие в 443 раза и ускорение инференса в 25 раз при сохранении более 90% точности.

Структура работы. Диссертация состоит из введения, пяти глав, заключения, библиографического списка из 37 источников и пяти приложений. Глава 1 содержит анализ предметной области и обзор существующих

решений. Глава 2 описывает проектирование архитектуры системы. Глава 3 посвящена реализации компонентов. Глава 4 представляет оптимизацию SmolVLA. Глава 5 описывает верификацию системы автоматизированными тестами.

ГЛАВА 1. Анализ предметной области и обзор существующих решений

1.1. Кооперативное визуальное восприятие в многоагентных системах

1.1.1. Постановка задачи и фундаментальный компромисс

Задача кооперативного восприятия формулируется следующим образом: группа агентов, каждый из которых оснащён собственными датчиками (камера, лидар, IMU), должна совместно построить более полное и устойчивое представление об окружающей среде, чем любой агент мог бы получить в одиночку. Ключевым практическим ограничением является *компромисс между качеством восприятия и стоимостью коммуникации*: передача сырых данных с высокой частотой быстро исчерпывает полосу пропускания, тогда как чрезмерное сжатие ведёт к потере перцептивно важной информации.

Обзорные работы систематизируют подходы к кооперативному восприятию по трём уровням обмена: ранний (сырые данные), поздний (готовые детекции) и промежуточный (признаки из скрытых слоёв) [1, 2]. Промежуточный уровень в настоящее время рассматривается как наиболее перспективный, поскольку позволяет существенно снизить объём передаваемых данных при незначительных потерях качества. Реальная ценность cooperative perception возникает не просто от «обмена данными», а от обмена компактными, согласованными и выборочно передаваемыми признаками [2].

1.1.2. Ключевые алгоритмические подходы

V2VNet [3] стал одной из первых работ, показавших практическую применимость промежуточного слияния: автомобили обмениваются сжатыми картами активаций, что позволяет обнаруживать объекты за препятствиями. Система достигает 30% прироста в mAP по сравнению с одиночным восприятием при ограниченной полосе пропускания.

When2com [4] ввёл принципиально новый вопрос: не только *чем* делиться, но и *когда* стоит коммуницировать. Предложенный механизм handshake позволяет агенту запрашивать информацию только тогда, когда

она действительно повышает качество восприятия, формируя динамические группы через граф коммуникации.

Where2comm [5] развивает идею пространственной выборки: агенты передают только перцептивно важные, пространственно разреженные фрагменты карты признаков, используя *spatial confidence maps*. Система демонстрирует до 1000-кратного сжатия объёма передаваемых данных при незначительном снижении качества.

V2X-ViT [6] переносит трансформерную архитектуру на задачу V2X-восприятия, вводя чередующиеся слои гетерогенного *self-attention* между агентами и *multi-scale window self-attention* внутри каждого агента.

OPV2V [7] и **CoBEVT** [8] представляют первый крупный открытый бенчмарк для V2V-восприятия и первый фреймворк для кооперативной генерации Bird's Eye View карт соответственно. Датасет **CoPeD** [9] — первый гетерогенный реальный датасет для многороботного кооперативного восприятия с воздушными и наземными роботами — фиксирует разрыв между автомобильной кооперативной перцепцией и задачами robotics.

1.2. Совместное картографирование и локализация

Параллельным направлением к cooperative perception является **Collaborative SLAM (C-SLAM)** — совместное построение карты и локализация несколькими роботами. Обзор [10] систематизирует свыше 100 работ и выделяет три фундаментальные проблемы: робастность (устойчивость к ошибкам ассоциации данных), управление коммуникацией и вычислительная масштабируемость при росте числа агентов.

Swarm-SLAM [11] представляет открытую децентрализованную C-SLAM систему, поддерживающую лидарные, стерео-, RGB-D и инерциальные сенсоры. Ключевое нововведение — техника приоритизации межроботных замыканий петель, позволяющая сократить объём передаваемых данных при ускорении сходимости. Тренд в направлении C-SLAM — движение от централизованных схем к разреженным децентрализованным, что согласуется с логикой масштабирования реальных многороботных флотов.

Для AgentsSwarm эти работы образуют теоретический мост между «обменом визуальной информацией» и «совместным выполнением миссий на общей карте»: система использует *occupancy map* и *waypoint graph* как общее

пространственное представление, доступное всем компонентам планирования.

1.3. Координация флота и стандарты интероперабельности

1.3.1. VDA5050 как промышленный стандарт

Стандарт **VDA5050** [12], разработанный VDA и VDMA, определяет открытый протокол коммуникации между флотом мобильных роботов (AGV/AMR) и центральной системой fleet control через MQTT-брокер и JSON-сообщения. Стандарт явно нацелен на решение проблемы *интероперабельности*: роботы разных производителей должны взаимодействовать с единым fleet manager без уникальных point-to-point интеграций.

Работа [13] рассматривает VDA5050 с промышленной перспективы: авторы из Robert Bosch GmbH анализируют задачи multi-agent decision making для реальных intralogistics-сценариев, где AMR работают рядом с людьми, legacy AGV и разнородным оборудованием. Для промышленного развёртывания уже недостаточно «уметь ездить»: нужен interoperable decision stack с поддержкой VDA5050 как единого языка взаимодействия.

1.3.2. OpenRMF и архитектурные аналоги

Open RMF [14] реализует централизованные функции: task queuing, conflict-free scheduling и fleet adapters для совместимости с разными роботами. Core-пакеты OpenRMF предоставляют не алгоритмы навигации, а middleware-логику — то же разделение ответственности, что реализовано в AgentsSwarm между Mission Control (планирование и граф карты) и Mission Dispatch (очередь и доставка миссий).

1.3.3. Behavior Trees как исполнимый формат миссий

Behavior Trees (BT) [15] стали доминирующим подходом к формализации миссионной логики в современной робототехнике: по сравнению с классическими FSM, BT лучше масштабируются при усложнении поведения и обеспечивают более явное разделение логики условий и действий. Mission Control NVIDIA использует BT как внутренний исполнимый формат для разложения высокоуровневых задач на последовательности действий роботов.

1.3.4. Облачная и туманная робототехника

FogROS2 [16] демонстрирует, что offloading вычислительно тяжёлых алгоритмов (SLAM, grasp planning) в облако даёт радикальное ускорение: снижение задержки SLAM на 50%, ускорение планирования захватов с 14 с до 1,2 с. **Robofleet** [17] решает задачу эффективной коммуникации в флоте с дедупликацией трафика и adaptive backpressure. Литература последовательно указывает на то, что наиболее практичны архитектуры, где низкоуровневое управление остаётся в робототехническом стеке (ROS 2, VDA5050 клиент), а облако берёт на себя планирование и семантическое reasoning — именно эту стратегию реализует AgentsSwarm.

1.4. Большие языковые модели в робототехнике

1.4.1. Смена парадигмы: от чат-бота к reasoning-агенту

Применение LLM в робототехнике прошло несколько этапов. Ранние работы использовали языковые модели как интерфейс «понимания команд». Переломным стало осознание, что LLM обладают commonsense reasoning и способностью к декомпозиции сложных задач. Обзоры [18, 19] классифицируют применение LLM по четырём слоям: коммуникация с человеком, перцептивная семантическая интерпретация, task planning и генерация действий. Для многоагентных систем выделяется отдельный класс задач: high-level coordination, task allocation и human-in-the-loop supervision [19].

1.4.2. Парадигма ReAct и tool use

ReAct [20] стал методологической основой для агентных систем нового поколения: LLM чередует генерацию рассуждений (reasoning traces) и действий (actions), что позволяет динамически корректировать план на основе наблюдаемых результатов. В отличие от одношагового prompting, ReAct создаёт замкнутый цикл *reasoning* → *action* → *observation* → *reasoning*.

SayCan [21] решает проблему «заземления» языкового плана в физические ограничения среды: LLM генерирует вероятности возможных действий, а модель аффордансов взвешивает их реализуемость (84% точности планирования, 74% успеха выполнения). **ProgPrompt** [22] вводит программноподобную структуру промптов, что существенно повышает воспроизводимость генерируемых планов.

Архитектура оркестратора AgentsSwarm следует той же логике: LLM работает как высокоуровневый диспетчер и планировщик поверх инструментов (MCP-серверы) и mission APIs, не находясь в servo-loop робота.

1.4.3. Безопасность и формальные ограничения LLM-планирования

SELP [23] предлагает три техники для генерации безопасных планов: голосование по эквивалентности, constrained decoding через LTL-формулы и доменное дообучение. Система показывает +10,8% к safety rate и +19,8% к эффективности для навигации БПЛА. Это направление актуально для AgentsSwarm: текущая система корректно разграничивает LLM-слой и mission-контур, однако формальная верификация LLM-выходов остаётся открытым вопросом.

1.4.4. LLM для координации роя

Swarm-GPT [24] исследует применение LLM для управления роем БПЛА: модель генерирует траектории, а алгоритм безопасного планирования фильтрует коллизии (100% безопасных траекторий после фильтрации против 21,65% напрямую от LLM). Ключевой принцип: *LLM как генератор идей, классический алгоритм как гарант безопасности*.

1.5. Foundation Models и Vision-Language-Action модели

RT-2 [25] продемонстрировал, что совместное обучение на данных роботов и интернет-данных позволяет VLA-модели переносить семантические знания в реальное управление роботом. **VoxPoser** [26] показал комплементарный подход: LLM генерирует код для составления 3D value maps, по которым планировщик синтезирует траектории манипулятора в zero-shot режиме. **OpenVLA** [27] — открытая модель на 7B параметров, поддерживающая fine-tuning через LoRA. **Open X-Embodiment** [28] — датасет из более чем 1M реальных траекторий от 22 типов роботов — стал важнейшим ресурсом для обобщённых VLA-моделей. π_0 [29] представляет следующий уровень — универсальный VLA на основе flow matching.

Параллельно с масштабированием идёт компактификация VLA для ресурсно ограниченных платформ. **SmolVLA** [30] (450M параметров) обучается на одном GPU и сохраняет производительность при значительно меньших требованиях. **TinyVLA** [31] с диффузионным декодером превос-

ходит OpenVLA при задержке инференса в 20 раз меньше. В контексте AgentsSwarm экспериментальный модуль SmolVLA Tools реализовал пайплайн оптимизации SmolVLA с результатами, описанными в главе 4.

1.6. Симуляционные среды и цифровые двойники

NVIDIA Isaac Sim [32] — фотореалистичная физически правдоподобная среда симуляции, построенная на Omniverse. Официальный tutorial по multi-robot navigation демонстрирует одновременную навигацию нескольких роботов с ROS 2 namespaces — это практически совпадает с логикой AgentsSwarm, где одна сцена обслуживает несколько роботов Carter. **Orbit (Isaac Lab)** [33] позиционирует Isaac Sim как унифицированный фреймворк для роботообучения. **Pegasus Simulator** [34] демонстрирует расширяемость Isaac Sim для специфических платформ (БПЛА с PX4).

Обзор по AI-driven цифровым двойникам [35] показывает, что основная сложность лежит не в симуляции как таковой, а в интероперабельности моделей и sim-to-real переносе. Для AgentsSwarm это означает честное признание: система закрывает симуляционный и orchestration-контур, но перенос на физический флот требует отдельного исследовательского этапа.

vLLM / PagedAttention [36] решает проблему эффективного использования GPU-памяти при serving LLM: алгоритм PagedAttention, вдохновлённый виртуальной памятью ОС, увеличивает пропускную способность в 2–4 раза. **Qwen2.5** [37] — семейство открытых instruction-following моделей, обученных на 18 трлн токенов. Выбор Qwen-Instruct как основной модели оркестратора обеспечивает баланс между качеством tool call и доступностью для самостоятельного развёртывания.

1.7. Выводы и результаты по главе 1

На основе проведённого анализа можно выделить несколько классов существующих систем и сопоставить с ними AgentsSwarm.

Академические прототипы кооперативного восприятия (V2VNet, Where2comm, CoBEVT) сосредоточены на алгоритмической эффективности feature fusion. Они не решают задачу интеграции восприятия с fleet management и не имеют исполнимого миссионного контура. AgentsSwarm дополняет этот класс: вместо нового алгоритма fusion предлагается архитекту-

ра, в которой кооперативное восприятие встроено в полный цикл управления флотом.

Промышленные fleet management системы (OpenRMF, VDA5050-совместимые стеки) решают задачи координации и интероперабельности, но не интегрируют семантическое LLM-планирование. AgentsSwarm добавляет этот слой через оркестратор с девятью специализированными агентами.

LLM-robotics прототипы (SayCan, Swarm-GPT, RT-2) демонстрируют применение языковых и мультимодальных моделей, но либо работают с одиноким роботом, либо не интегрированы с промышленными стандартами координации флота. AgentsSwarm соединяет LLM-слой с VDA5050-контуром.

Гибридная позиция AgentsSwarm — между академической литературой по collaborative perception, промышленными multi-robot middleware и emerging-работами по LLM-driven robotics — является как источником научного вклада, так и честным признанием ограничений: система не представляет новый state-of-the-art алгоритм, но демонстрирует воспроизводимую системную интеграцию, которая в литературе встречается значительно реже, чем изолированные алгоритмические вклады.

Открытыми вопросами остаются: формальные гарантии LLM-слоя (hallucination, constraint violations), sim-to-real перенос и бортовая автономность — направления, которые система AgentsSwarm обозначает через модуль SmolVLA Tools и архитектурные расширения, описанные в следующих главах.

ГЛАВА 2. Проектирование архитектуры системы AgentsSwarm

2.1. Требования к системе

Проектирование системы AgentsSwarm началось с формулировки функциональных и нефункциональных требований, определивших архитектурные решения на всех уровнях.

Функциональные требования:

1. Приём пользовательского запроса на естественном языке через веб-интерфейс.
2. Семантическая классификация запроса и выбор специализированного агента-исполнителя.
3. Высокоуровневое планирование миссии с декомпозицией на шаги и назначением целевых роботов.
4. Трансляция плана в VDA5050-совместимые команды через Mission Control и Mission Dispatch.
5. Отображение статуса выполнения миссии в реальном времени (streaming).
6. Поддержка параллельных пользовательских сессий с фоновой обработкой задач.
7. Хранение истории диалогов и персональной памяти пользователя.
8. Генерация дополнительных артефактов: изображений, презентаций, исследовательских отчётов.

Нефункциональные требования:

- *Воспроизводимость*: все компоненты упакованы в Docker-контейнеры с фиксированными версиями зависимостей.
- *Интероперабельность*: использование открытых стандартов (VDA5050, MCP, ROS 2, OpenAI-compatible API).
- *Масштабируемость*: горизонтальное масштабирование агентов через Celery-воркеры.
- *Наблюдаемость*: структурированные события в Redis Stream, трассировка агентного маршрута в UI.

Ограничения: система развёртывается на трёх облачных машинах (Selectel, MTS Cloud), что определяет распределение компонентов по узлам и требования к сетевому взаимодействию.

2.2. Общая микросервисная архитектура

Система декомпозирована на восемь самостоятельных компонентов, взаимодействующих через REST API, WebSocket, MQTT и Model Context Protocol.

1. **Interface** — пользовательский веб-портал (React + FastAPI + Celery + RabbitMQ + Redis Streams + PostgreSQL + MinIO). Принимает запросы, маршрутизирует к агентам, стримит ответы через WebSocket.
2. **Orchestrator** — главный интеллектуальный сервис (FastAPI + OpenAI Agents SDK). Содержит девять специализированных агентов, взаимодействует с роботическим контуром через MCP-серверы.
3. **vLLM Service** — сервис инференса LLM в режиме Data Parallel ($2 \times$ Tesla V100). Предоставляет OpenAI-compatible API.
4. **Mission Control** — форк NVIDIA, реализующий VDA5050-совместимый fleet manager с Behavior Tree планировщиком и cuOpt-оптимизацией.
5. **Mission Dispatch** — форк NVIDIA, реализующий очередь миссий и VDA5050-обмен через MQTT.
6. **Isaac Sim** — среда симуляции NVIDIA с роботами Carter в headless-режиме, интегрированная с ROS 2 Jazzy.
7. **ROS 2 Workspace** — форк с дополнительными пакетами навигации, VDA5050 клиентом, rosbridge WebSocket.
8. **SmolVLA Tools** — экспериментальный модуль оптимизации VLA-модели.

Слои системы: *воспринимающий* (Isaac Sim, ROS 2), *планирующий* (Orchestrator, vLLM, MCP-серверы) и *исполняющий* (Mission Control, Mission Dispatch, VDA5050 клиент робота).

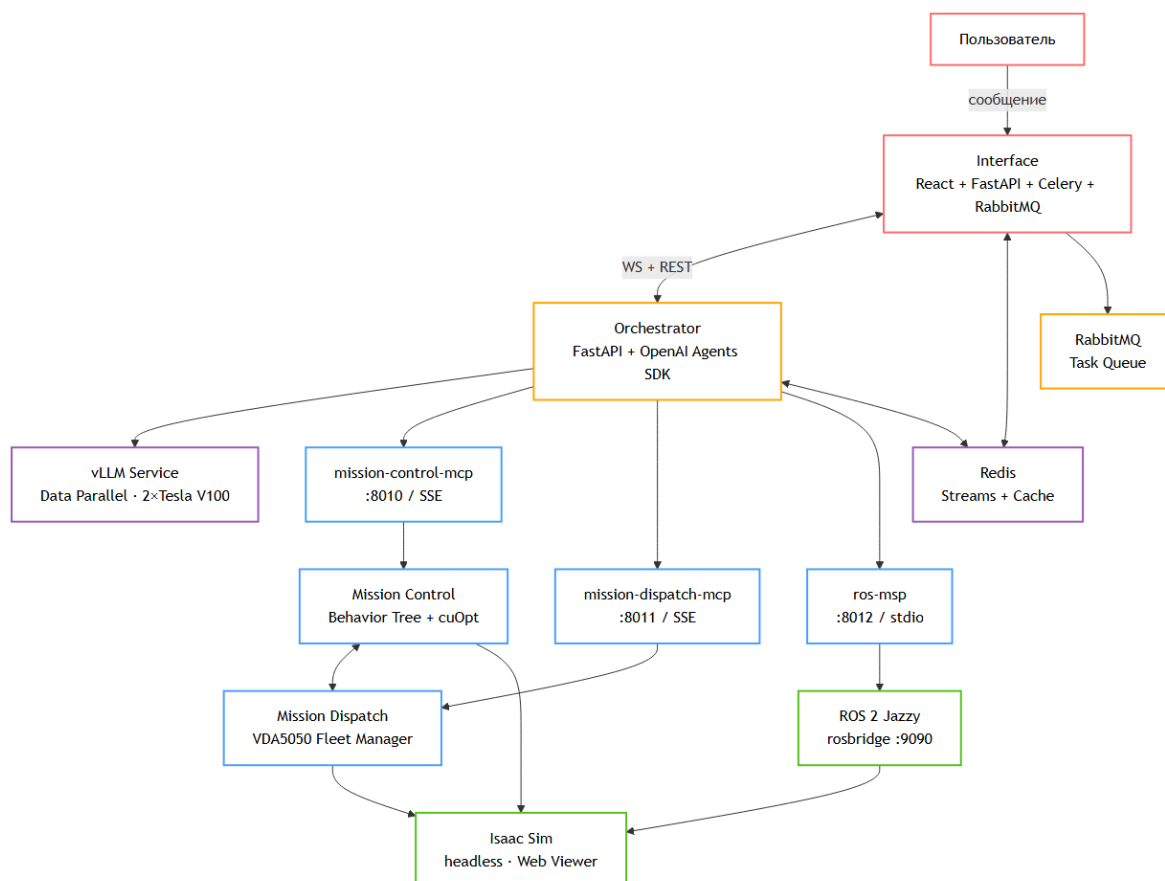


Рис. 2.1. Общая схема взаимодействия компонентов системы AgentsSwarm

2.3. Протокол взаимодействия агентов: Model Context Protocol

Model Context Protocol (MCP) — открытый стандарт для взаимодействия LLM-агентов с внешними инструментами и данными. Протокол определяет архитектуру клиент-сервер, где MCP-клиент (агент) вызывает tools, resources и prompts, предоставляемые MCP-сервером. Транспортный слой поддерживает два режима: SSE (Server-Sent Events) для HTTP-развёртывания и stdio для локальных процессов.

В системе AgentsSwarm реализованы три MCP-сервера с чёткими зонами ответственности:

- `mission-control-mcp` — инструменты для работы с картой, waypoint-графом и статусами роботов (Mission Control API);
- `mission-dispatch-mcp` — инструменты для создания, управления и мониторинга миссий (Mission Dispatch API);
- `ros-msp` — инструменты для публикации в ROS 2 топики и чтения из них через rosbridge WebSocket.

MCP обеспечивает строгое разделение ответственности: агентный слой

не зависит от конкретной реализации роботического контура и взаимодействует с ним исключительно через стандартизованный tool-интерфейс. Это позволяет заменять или расширять роботический контур без модификации кода оркестратора.

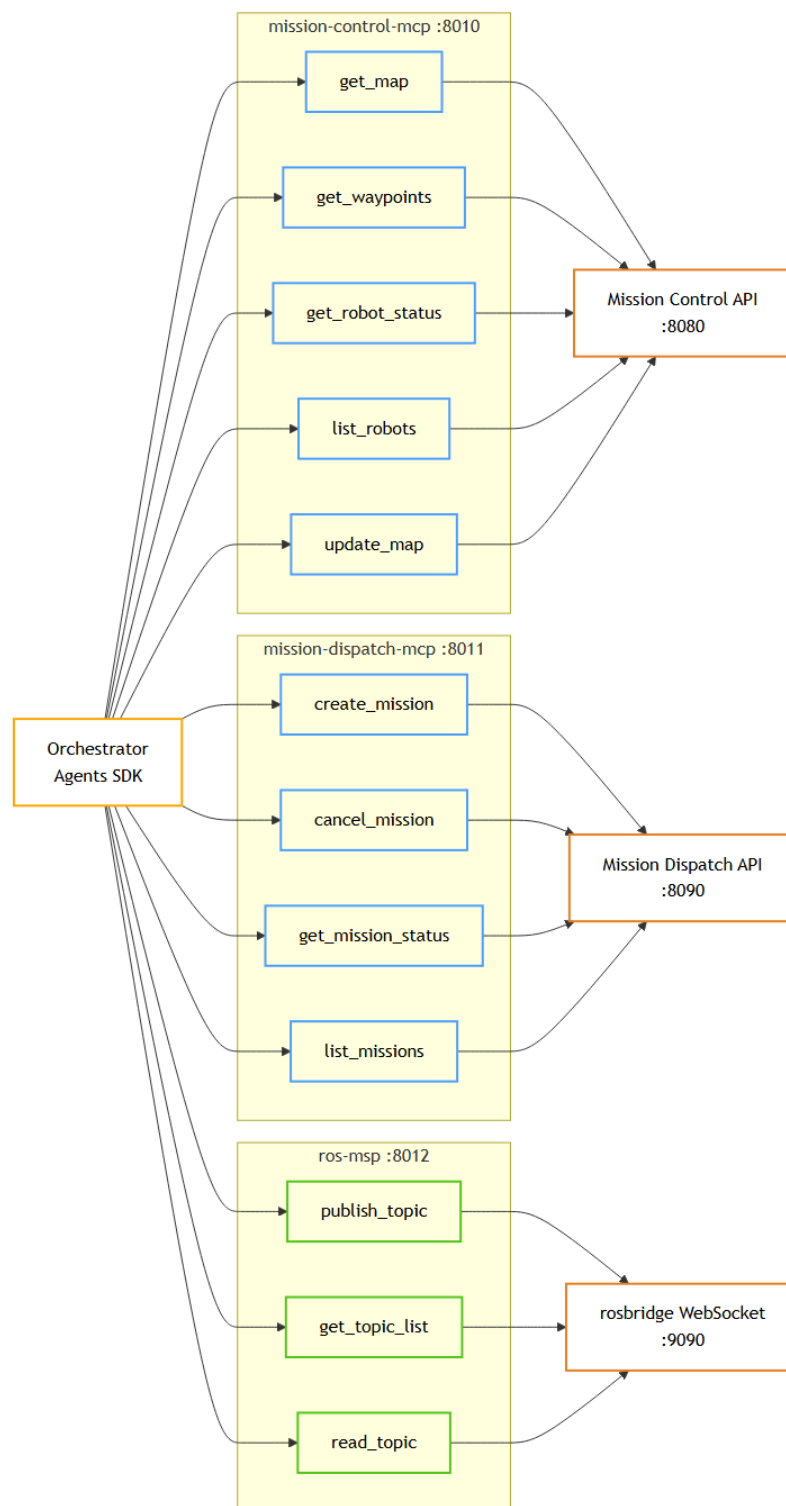


Рис. 2.2. Три MCP-сервера системы AgentsSwarm и их зоны ответственности

2.4. Протокол управления флотом: VDA5050

Стандарт VDA5050 [12] определяет JSON-структуры сообщений, передаваемых через MQTT-брокер:

- `order` — задание для робота (последовательность узлов графа и рёбер с действиями);
- `state` — текущее состояние робота (позиция, статус, ошибки, уровень заряда);
- `connection` — статус подключения робота к `fleet manager`.

Маршрут команды в AgentsSwarm:

Orchestrator → MCP-сервер → Mission Dispatch API → MQTT → VDA5050 клиент → Isaac Sim

Mission Control принимает высокоуровневую задачу, строит Behavior Tree, запрашивает оптимальный маршрут у cuOpt и формирует VDA5050 order для Mission Dispatch. Mission Dispatch публикует order в MQTT-топик, VDA5050 клиент робота в Isaac Sim подписан на этот топик и исполняет команду.

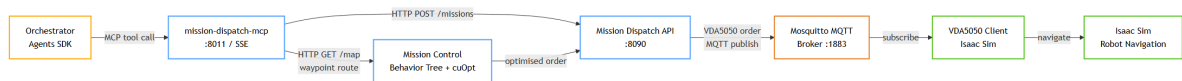


Рис. 2.3. Маршрут команды: Orchestrator → MCP → Mission Dispatch → VDA5050 → робот

2.5. Агентная архитектура оркестратора

Агентная архитектура оркестратора построена по иерархическому принципу: входящий запрос обрабатывается Router Agent, классифицирующим его по восьми категориям, после чего управление передаётся через handoff-механизм специализированному агенту.

Router Agent классифицирует запросы по категориям: *general* (общие вопросы), *mission* (команды роботам), *navigation* (задачи навигации), *charging* (управление зарядкой), *patrol* (патрульные маршруты), *inspection* (инспекция объектов), *fleet_ops* (операции с флотом), *swarm* (координация роя). Классификация выполняется через LLM с детерминированными правилами приоритетов.

MissionPlanner реализует двухфазный алгоритм: на первой фазе строит план в виде последовательности именованных шагов с указанием

`target_robots` и зависимостей; на второй — последовательно исполняет шаги, вызывая инструменты MCP-серверов.

MapAnalyst получает PNG-снимок карты от Mission Control, накладывает позиции роботов и передаёт изображение в vision-LLM для пространственного анализа сцены.

Остальные агенты — Navigation, Charging, Patrol, Inspection, FleetOps, SwarmCoordinator, General — специализированы под конкретные типы задач и содержат профилированные системные промпты.

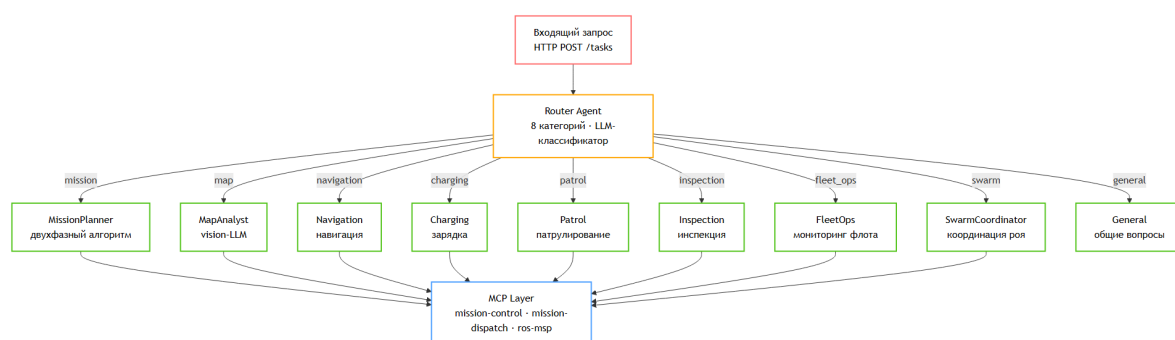


Рис. 2.4. Иерархия агентов оркестратора: Router Agent и специализированные агенты

2.6. Событийно-ориентированная потоковая модель

Асинхронная потоковая модель Interface построена на Redis Stream как шине событий. Агент-воркер публикует события в поток `chat:{thread_id}:stream` командой `XADD`; WebSocket-слой читает поток командой `XREAD` с блокирующим ожиданием и проталкивает события клиенту.

Жизненный цикл задачи: *pending* → *running* (первый чанк) → *completed* или *error*. Типы событий: `stream_chunk`, `agent_reply`, `routing_event`, `tool_event`, `error`.

Параллельный канал — WebSocket оркестратора (`/ws/task/{task_id}`), по которому `SwarmOrchestratorAgent` получает события от оркестратора и транслирует их в Redis Stream Interface, обеспечивая сквозную прозрачность агентного маршрута для пользователя.

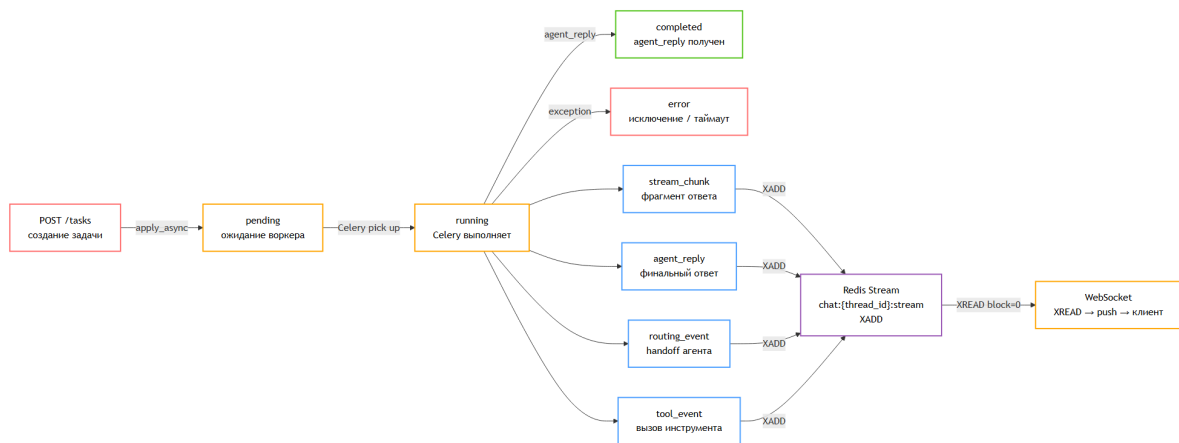


Рис. 2.5. Жизненный цикл задачи (pending → completed) и потоковая модель событий Redis Streams

2.7. Инфраструктура развёртывания

Три машины разной конфигурации обслуживают различные подсистемы системы AgentsSwarm (Табл. 2.1).

Таблица 2.1. Технические характеристики вычислительной инфраструктуры

Машина	Провайдер	Конфигурация	Назначение
Симуляция	Selectel	4 vCPU, 16 ГБ RAM, RTX 4090 (24 ГБ VRAM), Ubuntu 24.04	Isaac Sim, ROS 2, Mission Control, Mission Dispatch
Микросервисы	Selectel	4 vCPU, 8 ГБ RAM, 128 ГБ SSD, Ubuntu 24.04	Interface, Orchestrator, Redis, RabbitMQ, MinIO, PostgreSQL
LLM-кластер	MTS Cloud	2 ноды × Tesla V100-PCIE 32 ГБ VRAM, Ubuntu 24.04	vLLM Data Parallel (Qwen2.5-Instruct)

Межсервисное взаимодействие: HTTP/REST (оркестратор ↔ MCP, Interface ↔ оркестратор), WebSocket (стриминг событий), MQTT (VDA5050), RPC gRPC (vLLM Data Parallel, порт 13345).

2.8. Выводы и результаты по главе 2

Спроектированная архитектура AgentsSwarm обеспечивает чёткое разделение ответственности между компонентами: пользовательский слой (Interface) изолирован от роботического контура (Mission Control/Dispatch/Isaac Sim) через двухступенчатый агентный барьер (Interface-агенты → Orchestrator-агенты → MCP-серверы).

Принципиальной новизной по сравнению с рассмотренными аналогами является интеграция LLM-слоя с промышленным VDA5050-контуром через стандартизованный MCP-протокол. OpenRMF [14] и аналогичные fleet management системы требуют формального задания задачи через API; AgentsSwarm принимает произвольный запрос на естественном языке и транслирует его в исполнимую последовательность VDA5050-команд. Событийно-ориентированная потоковая модель на Redis Streams обеспечивает наблюдаемость сквозного контура выполнения задачи для пользователя в реальном времени.

ГЛАВА 3. Реализация компонентов системы

3.1. Симуляционная среда: NVIDIA Isaac Sim и ROS 2 Workspace

Симуляционная среда построена на базе NVIDIA Isaac Sim 4.5 в headless-режиме с Web Viewer для удалённого наблюдения. Сцена Carter Warehouse содержит несколько роботов Carter с уникальными ROS 2-пространствами имён (namespaces), что позволяет независимо управлять каждым роботом через единый ROS 2 DDS-мост.

Запуск осуществляется через Docker Compose стек: контейнер Isaac Sim монтирует GPU, обеспечивает доступ к драйверам CUDA и публикует Web Viewer на порту 8211. Конфигурация Action Graph задаёт параметры синтетической камеры, публикует RGB- и depth-изображения в ROS 2-топики и запускает `isaac_ros_mission_client` — официальный пакет NVIDIA, реализующий VDA5050 клиент для флота роботов.

ROS 2 Jazzy workspace собирается из форка `isaac_ros_common`, дополненного пакетами для отображения occupancy map, публикации costmap и rosbridge-сервером. Rosbridge WebSocket (порт 9090) обеспечивает доступ к ROS 2-топикам для MCP-сервера без необходимости установки полного стека ROS 2 на машине оркестратора.

Occupancy map генерируется из ортогонального вида сцены Isaac Sim, сохраняется в формате PNG и обновляется в Mission Control через endpoint `/map/update_robot`. На основе карты Mission Control строит `waypoint graph` — ориентированный граф навигационных узлов, по которому `cuOpt` рассчитывает оптимальные маршруты.

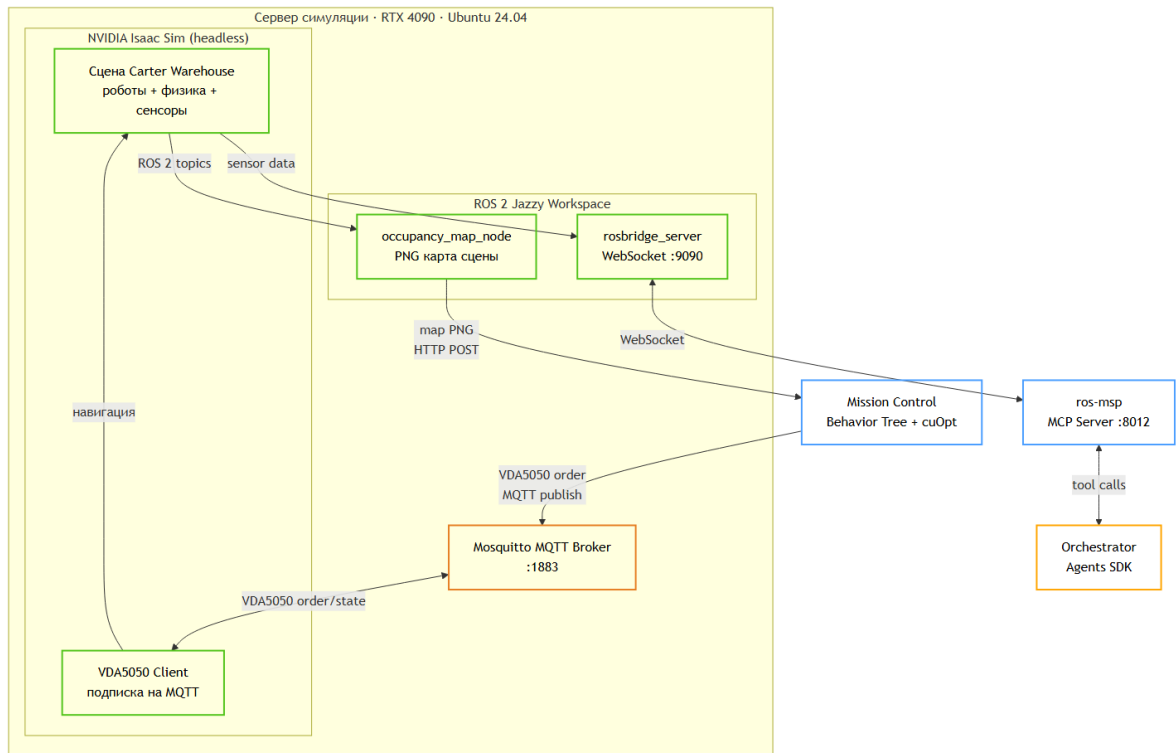


Рис. 3.1. Схема интеграции Isaac Sim, ROS 2 и VDA5050 в симуляционном контуре

[Рис. 3.2. Occupancy map и waypoint graph сцены Carter Warehouse]

3.2. Mission Control: форк NVIDIA и авторские доработки

Официальный репозиторий NVIDIA Mission Control реализует централизованный fleet manager для VDA5050-совместимых роботов с Behavior Tree планировщиком и поддержкой cuOpt для оптимизации маршрутов. При развёртывании оригинального кода были выявлены критические ошибки.

Проблема 1: несоответствие endpoint обновления карты. Оригинальный код вызывал `/push_map`, тогда как Mission Control API использует `/map/update_robot` и `/map/metadata`. Исправление потребовало анализа HTTP-трафика между компонентами и корректировки маршрутизации внутри Mission Control.

Проблема 2: устаревший синтаксис Pydantic v1. Модуль `waypoint_selection_ui` использовал декораторы-валидаторы, несовместимые с Pydantic v2. Исправление состояло в замене на актуальный синтаксис `@field_validator`.

Стабилизация Docker-стека включала корректировку порядка запуска зависимых сервисов (cuOpt, Mosquitto, PostgreSQL) и проверку health-check проб для корректной инициализации. После всех исправлений Mission

Control успешно принимает задания от оркестратора, строит Behavior Tree и публикует VDA5050 order в Mosquitto.

3.3. Mission Dispatch: форк NVIDIA и авторские доработки

Mission Dispatch реализует очередь миссий и двустороннее VDA5050-взаимодействие между fleet manager и роботами. В оригинальном коде обнаружена проблема с legacy endpoint в пакете `packages/controllers/server`: запросы к устаревшему маршруту возвращали 404, нарушая сквозное взаимодействие.

Дополнительно выявлена некорректная конфигурация MQTT-хоста: Mission Dispatch использовал жёстко заданный `localhost` вместо имени сервиса Docker Compose (`mosquitto`), что препятствовало работе в контейнеризованном окружении. После исправления двух ошибок проведена сквозная верификация цепочки:

Isaac Sim → VDA5050 клиент → Mosquitto → Mission Dispatch → Mission Control → Behavior Tree → VDA5050 order → Isaac Sim

3.4. MCP-серверы: глубокая переработка официальных реализаций

Три MCP-сервера обеспечивают агентам оркестратора доступ к роботическому контуру через стандартизованный tool-интерфейс. Исходные реализации официальных MCP-серверов NVIDIA представляли незавершённые прототипы с заглушечными endpoints и синхронной моделью вызовов.

Переход на асинхронную модель. Все инструменты переписаны на `async def` с `httpx.AsyncClient` для HTTP и `websockets` для `rosbridge`-соединений. Это обеспечивает корректную работу с `event loop` без блокировки при параллельных вызовах агентов.

mission-control-mcp предоставляет инструменты: `get_map` (PNG-карта), `get_waypoints` (навигационные узлы), `get_robot_status`, `list_robots`, `update_map`. Реализованы заглушечные endpoints для корректной инициализации Mission Control.

mission-dispatch-mcp предоставляет: `create_mission`, `cancel_mission`, `get_mission_status`, `list_missions`. Исправлены несовместимые endpoint-адреса.

ros-mcp предоставляет: `publish_topic`, `get_topic_list`, `read_topic`.
Сервер устанавливает постоянное WebSocket-соединение с `rosbridge` при инициализации.

Динамический выбор транспорта реализован через переменную окружения `MCP_TRANSPORT`: при значении `sse` сервер запускается как HTTP-приложение; при `stdio` — как локальный процесс со стандартными потоками. Это позволяет использовать один код для локального тестирования и production-развёртывания.

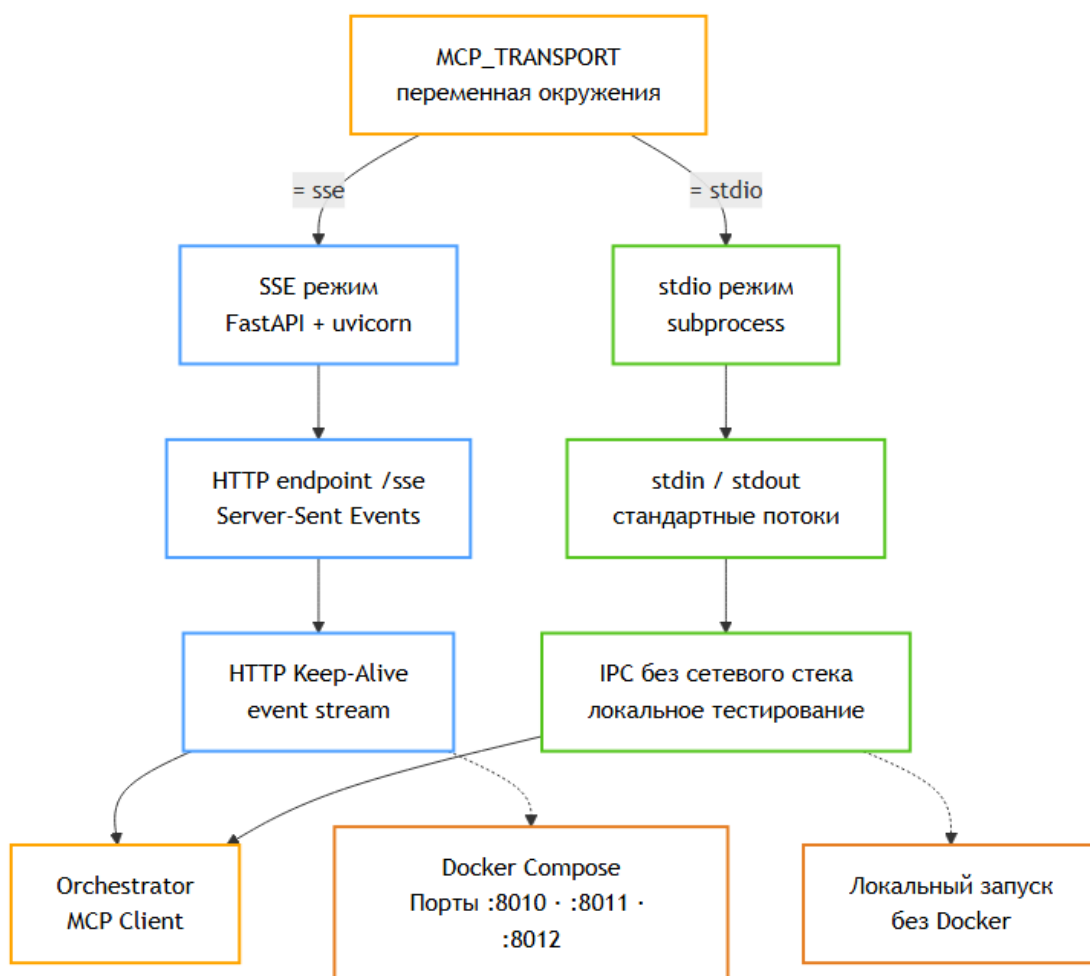


Рис. 3.2. Динамический выбор транспорта MCP: SSE (HTTP) и stdio режимы через переменную окружения

3.5. vLLM Service: авторский сервис инференса

Авторский vLLM-сервис обеспечивает инференс `Qwen2.5-Instruct` в режиме `Data Parallel` на двух физически разных узлах.

Архитектура Data Parallel: нода 0 (координатор, `rank 0`) принимает HTTP-запросы, токенизирует промпт и рассылает тензоры на ноду 1 (воркер,

rank 1) через gRPC-канал на порту 13345. Обе ноды независимо вычисляют forward pass по своей копии модели; координатор агрегирует логиты и генерирует следующий токен. Суммарно доступно 64 ГБ VRAM, что позволяет развернуть 32B-параметрическую модель с заполнением $\approx 90\%$ памяти.

Конфигурация нод задаётся через `.env.node0` и `.env.node1`: параметры `VLLM_HOST`, `VLLM_PORT`, `VLLM_GPU_MEMORY_UTILIZATION (0,90)`, `TENSOR_PARALLEL_SIZE=1`, `PIPELINE_PARALLEL_SIZE=2`.

Скрипты `deploy.sh` (Linux) и `deploy.bat` (Windows) автоматизируют: запуск Docker-контейнера vLLM с монтированием GPU и кэша модели; ожидание готовности `/health`; smoke-тест с тестовым промптом. Сервис полностью совместим с OpenAI Python SDK — требуется лишь установить `base_url` и `api_key="EMPTY"`.

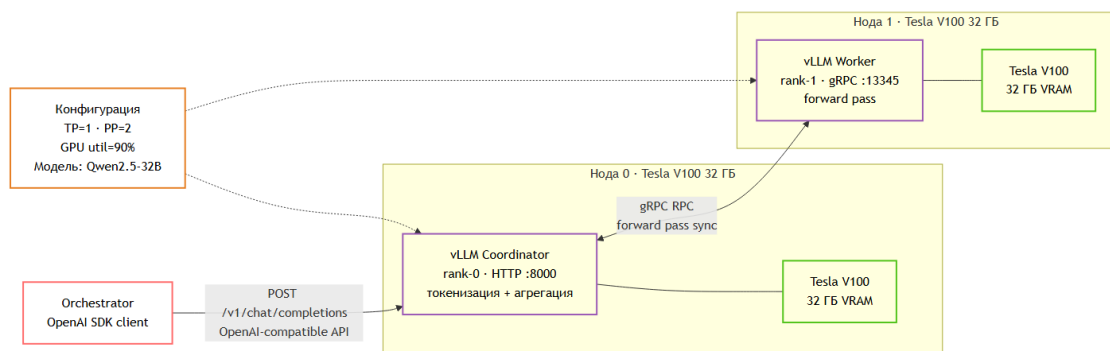


Рис. 3.3. Архитектура vLLM Data Parallel: координатор (rank-0) и воркер (rank-1), связанные через gRPC

3.6. Orchestrator: главный интеллектуальный микросервис

Оркестратор — основной интеллектуальный сервис системы AgentsSwarm, реализованный на FastAPI с lifespan-менеджером (версия v0.5.6, 56 коммитов).

Структура приложения. FastAPI-приложение организовано по роутерам: `/health`, `/agents`, `/sessions`, `/tasks`, `/ws`. Зависимости (SessionManager, StreamCollector, MCP-клиенты) регистрируются через Depends и инициализируются при старте lifespan.

Агентная архитектура (9 агентов). Все агенты наследуют от базового класса `BaseAgent` и конфигурируются через JSON-файлы промптов:

- *Router* — классификация запросов по 8 категориям;
- *MissionPlanner* — двухфазный алгоритм планирования и исполнения;

- *MapAnalyst* — vision-LLM для пространственного анализа карты;
- *Navigation, Charging, Patrol, Inspection* — специализированные агенты навигационных задач;
- *FleetOps* — мониторинг и операции с флотом;
- *SwarmCoordinator* — параллельная координация нескольких роботов.

Интеграция OpenAI Agents SDK. Вызов `Runner.run_streamed(agent, messages, context)` возвращает асинхронный итератор событий. `StreamCollector` присваивает каждому событию порядковый номер (`seq`), буферизует и публикует в WebSocket `/ws/task/{task_id}` с гарантией доставки по `seq`-номеру.

SessionManager хранит историю диалога в Redis с TTL 24 часа. Кэш результатов MCP-инструментов снижает число повторных HTTP-запросов к Mission Control.

Guardrails реализованы как слой валидации: входящий запрос проверяется на допустимую категорию, выходящий ответ — на соответствие схеме. Guardrails работают в silent fallback-режиме: невалидный ответ заменяется стандартным сообщением без прерывания сессии.

Retry с экспоненциальным backoff (максимум 3 попытки, начальный интервал 1 с) применяется для всех внешних вызовов (MCP, vLLM, Mission Control API).

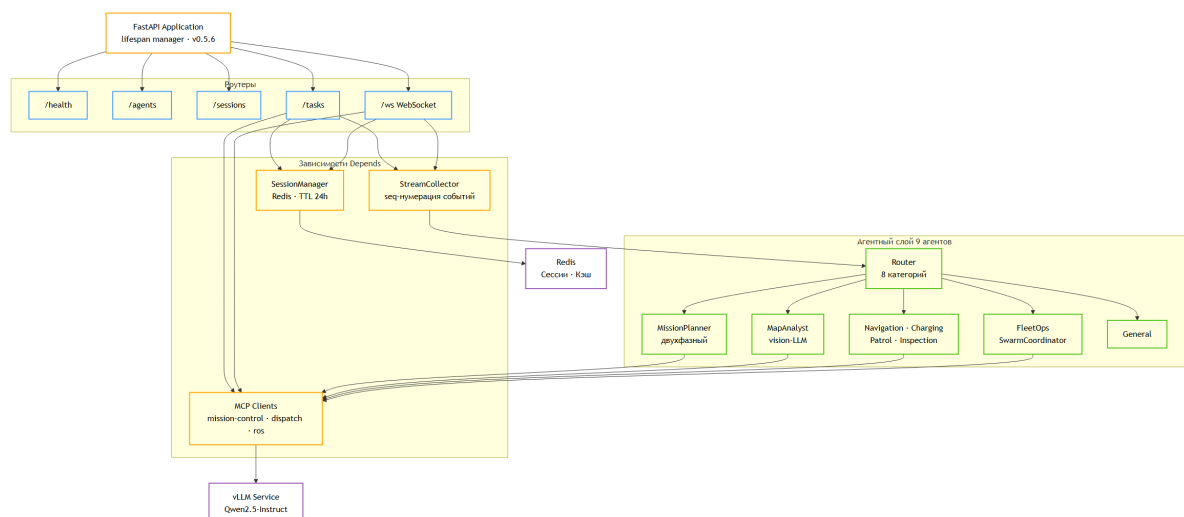


Рис. 3.4. Структура FastAPI-приложения оркестратора: роутеры, зависимости и агентный слой

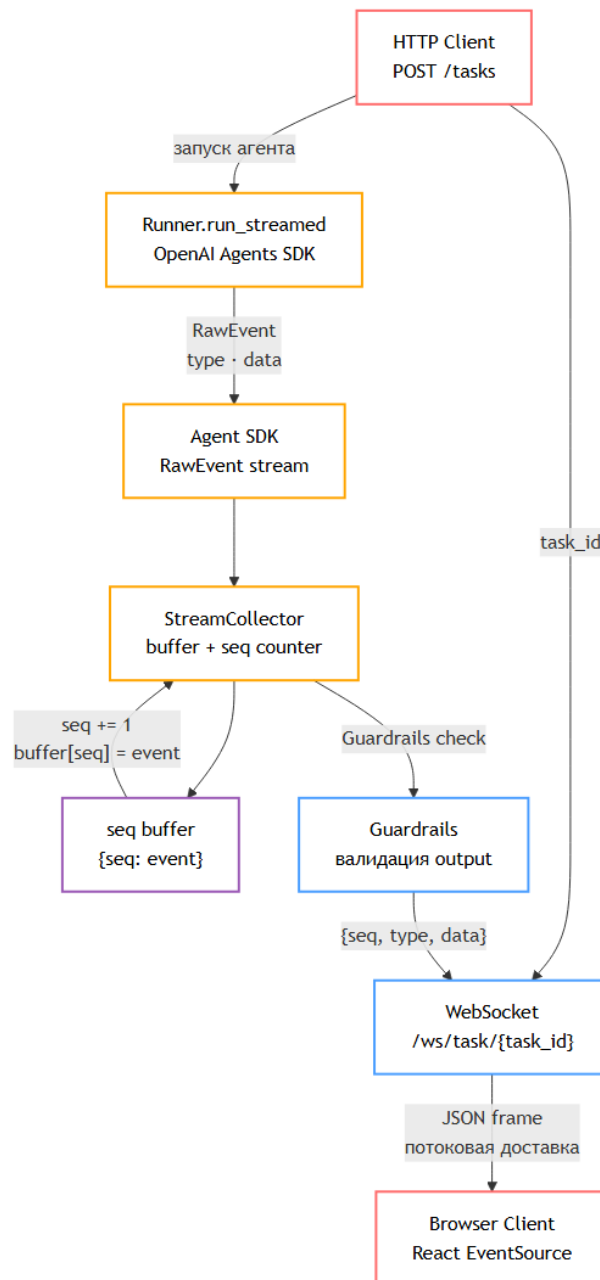


Рис. 3.5. StreamCollector: seq-нумерация событий Agents SDK и WebSocket push-доставка

3.7. Interface: веб-платформа управления

Interface — полнофункциональный веб-портал для взаимодействия пользователя с системой AgentsSwarm (v0.5.0, 168 коммитов). Компонент разработан полностью с нуля.

3.7.1. Общая архитектура и стек

Бэкенд реализован по clean architecture: слои *domain* (сущности и интерфейсы), *application* (ChatService, AgentExecutionService, JobOrchestrator), *infrastructure* (PostgreSQL/SQLAlchemy, Redis, MinIO), *api* (FastAPI роутеры,

WebSocket-слой). Docker Compose предоставляет dev-стек (hot-reload) и prod-стек (Nginx + Gunicorn).

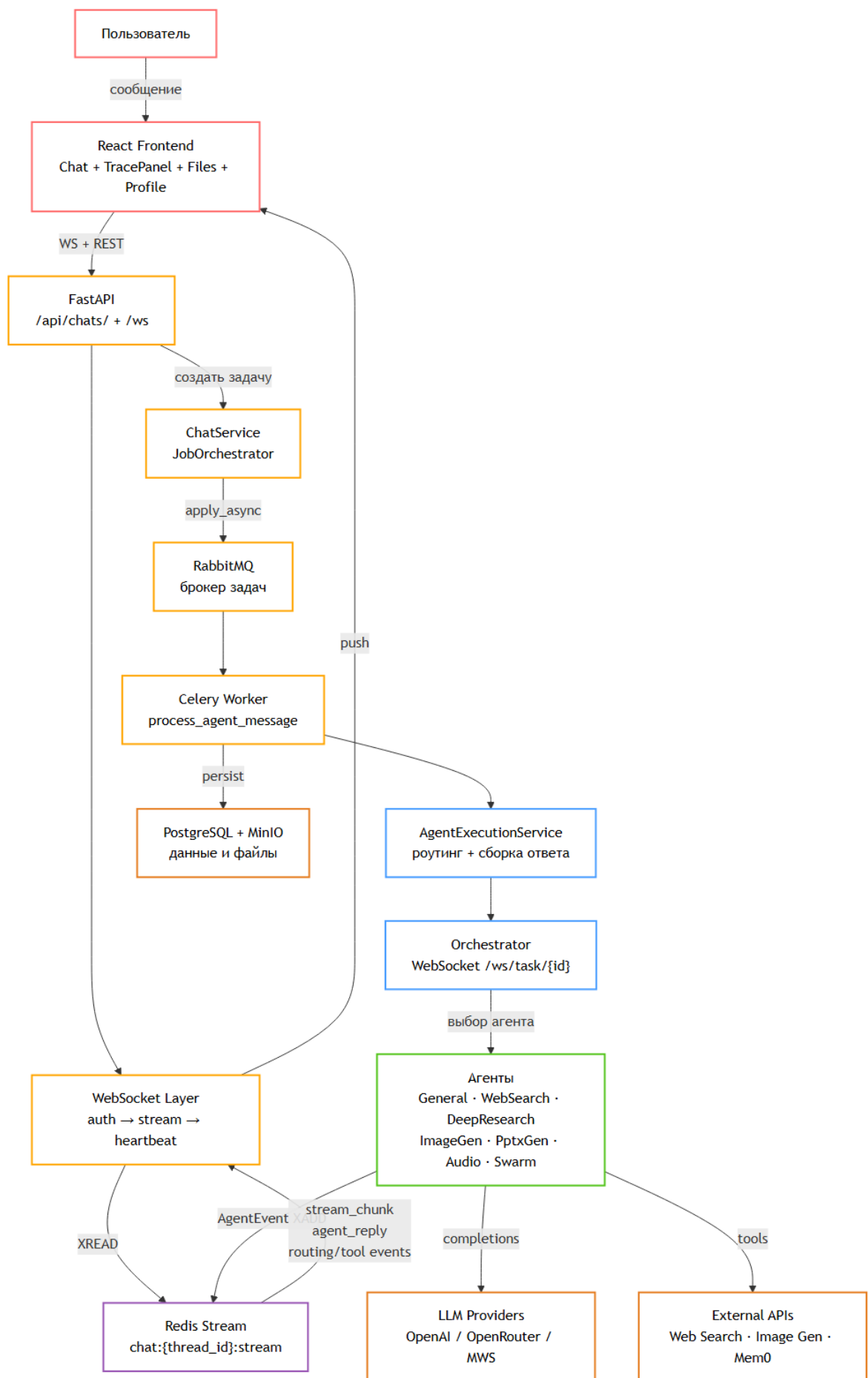


Рис. 3.6. Архитектура компонента Interface: React-фронтенд, FastAPI-бэкенд, Celery-воркеры и Redis Streams

3.7.2. Backend: агенты и обработка задач

JobOrchestrator маршрутизирует задачу к одному из семи агентов (Табл. 3.1):

Таблица 3.1. Специализированные агенты компонента Interface

Агент	Описание
General	Универсальный LLM-чат (Qwen2.5-Instruct)
WebSearch	Поиск в интернете с агрегацией результатов
DeepResearch	Многоитерационное глубокое исследование темы
ImageGen	Генерация изображений через внешний Image API
PptxGen	Создание PPTX-презентаций по шаблону
AudioTranscribe	Транскрипция аудио через Whisper
SwarmOrchestrator	Проксирование запросов к Orchestrator через WebSocket

Celery-задача `process_agent_message` запускается при получении сообщения. В ходе выполнения агент публикует события в Redis Stream `chat:{thread_id}:stream` через XADD.

3.7.3. Потокковая модель: Redis Streams и WebSocket

WebSocket-обработчик в FastAPI читает Redis Stream через XREAD с блокирующим ожиданием (`block=0`) и проталкивает каждое событие клиенту. Heartbeat-механизм каждые 15 с отправляет пинг для поддержания соединения.

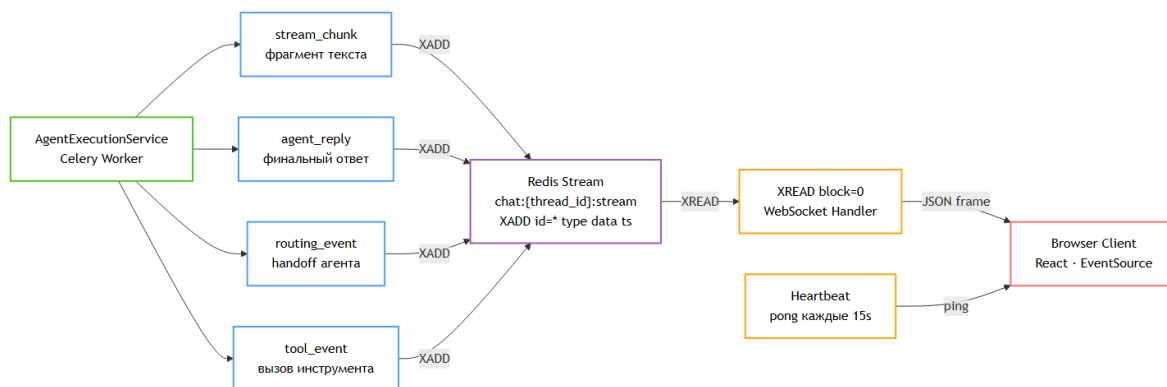


Рис. 3.7. Схема Redis Stream: XADD (агент) → XREAD (WebSocket-слой) → WebSocket → КЛИЕНТ

3.7.4. Frontend: страницы интерфейса

React-приложение на Chakra UI реализует четыре основных страницы:

- *Chat* — список диалогов, поле ввода с Markdown-рендером, потоковые фрагменты, загрузка файлов, выбор агента;
- *TracePanel* — timeline событий агентного маршрута: тип события, агент-источник, инструмент, временная метка;
- *Files* — галерея артефактов (изображения, PPTX, аудио) с предпросмотром и прямой ссылкой MinIO;
- *Profile / Memory* — управление персональной памятью пользователя (интеграция Mem0).

3.7.5. Демонстрация рабочего воркфлоу

В записанном демо показан полный сквозной сценарий: пользователь вводит запрос о навигации робота в Chat с выбором агента SwarmOrchestrator; Interface создаёт Celery-задачу, SwarmOrchestratorAgent устанавливает WebSocket-соединение с оркестратором; оркестратор выполняет Router → MissionPlanner → Navigation pipeline, вызывает MCP-инструменты, направляет робота Carter в заданную точку; результат стримится через Redis Stream и отображается в Chat и TracePanel в реальном времени.

[Рис. 3.9. Скриншот Chat: диалог с агентом SwarmOrchestrator]

[Рис. 3.10. Скриншот TracePanel: трассировка агентного маршрута]

[Рис. 3.11. Скриншот Files: галерея сгенерированных артефактов]

3.8. Выводы и результаты по главе 3

В данной главе описана реализация всех восьми компонентов AgentsSwarm. Ключевые технические решения и их обоснование:

- *Асинхронная модель MCP-серверов* — требование совместимости с event loop Agents SDK и параллельными вызовами инструментов;
- *Data Parallel vLLM* — единственная возможность уместить 32B-параметрическую модель на доступных Tesla V100;
- *Clean architecture Interface* — требование к независимому расширению набора агентов без модификации инфраструктурного кода;
- *Redis Streams для стриминга* — гарантированная доставка событий с возможностью переигрывания из истории.

Сводная таблица реализованных компонентов приведена в Табл. 3.2.

Таблица 3.2. Компоненты системы AgentsSwarm: версии и объём разработки

Компонент	Версия	Коммиты	Стек
Interface	v0.5.0	168	React, FastAPI, Celery, Redis, PostgreSQL, MinIO
Orchestrator	v0.5.6	56	FastAPI, OpenAI Agents SDK, Redis
vLLM Service	v0.2.9	—	vLLM, Docker, Qwen2.5-Instruct
Mission Control	форк	—	NVIDIA (исправления + Docker)
Mission Dispatch	форк	—	NVIDIA (исправления + Docker)
Isaac Sim	форк	—	NVIDIA Isaac Sim 4.5, ROS 2 Jazzy
ROS Workspace	2 форк	—	ROS 2 Jazzy, rosbridge, VDA5050 клиент
SmolVLA Tools	v0.1.0	—	PyTorch, LeRobot, ONNX

ГЛАВА 4. Оптимизация модели SmolVLA для бортового инференса

4.1. Постановка задачи и мотивация

Текущая архитектура AgentsSwarm использует облачный LLM для высокоуровневого planning и reasoning. Это вводит два фундаментальных ограничения: *задержка* между запросом агента и ответом LLM (при загруженном кластере — от 1 до нескольких секунд); *зависимость от сети* — при нестабильном соединении с облачным кластером роботический контур не может получить инструкции.

Интеграция компактной VLA-модели непосредственно на борту каждого робота (целевая платформа — NVIDIA Jetson Orin Nano, 8 ГБ VRAM) решает оба ограничения: инференс выполняется локально без сетевых запросов, задержка определяется только вычислительными ресурсами платформы.

SmolVLA [30] — компактная VLA-модель на 450М параметров от HuggingFace/LeRobot — была выбрана как наиболее перспективный кандидат для бортового развёртывания благодаря открытым весам, документированному API LeRobot и наличию асинхронного inference stack.

4.2. Архитектура SmolVLA и исходные характеристики

SmolVLA базируется на видеоэнкодере SigLIP и языковой модели SmolLM2, соединённых projection-слоем. На вход поступают RGB-изображение наблюдения и языковая инструкция; на выходе — последовательность действий манипулятора (7-мерный вектор: позиции 6 суставов и захват). Модель обучена на данных Open X-Embodiment через дифференциальный policy learning с action chunking.

Для верификации исходной точности проведена валидация Teacher-модели на датасете `lerobot/libero` (Табл. 4.1).

Таблица 4.1. Базовые метрики Teacher-модели SmolVLA на lerobot/libero

Метрика	Значение	Комментарий
MSE	0,1782	Среднеквадратичная ошибка действий
R^2	0,8412	Коэффициент детерминации
Параметры	450M	FP32, 6 ГБ VRAM
Латентность инференса	450 мс	RTX 4090, FP32

4.3. Пайплайн оптимизации

Пайплайн состоит из четырёх последовательных этапов.

Этап 1: Knowledge Distillation (KD). Student-модель строится как уменьшенная копия Teacher с коэффициентом `student_ratio` (отношение числа attention heads и скрытых размерностей к базовым). Функция потерь:

$$\mathcal{L}_{KD} = \alpha \cdot \mathcal{L}_{MSE} + (1 - \alpha) \cdot \mathcal{L}_{KL} + \lambda \cdot \mathcal{L}_{AT}, \quad (4.1)$$

где $\mathcal{L}_{MSE}$ — среднеквадратичная ошибка между выходами Student и Teacher, $\mathcal{L}_{KL}$ — дивергенция Kullback–Leibler смягчённых вероятностей с температурой $T = 3,0$, $\mathcal{L}_{AT}$ — ошибка передачи карт внимания (attention transfer loss). Гиперпараметры: $\alpha = 0,7$, $\lambda = 0,1$, 10 эпох обучения.

Этап 2: Mixed Precision Inference (FP16). Применяется `torch.cuda.amp` с автоматическим управлением precision: веса остаются в FP32, активации вычисляются в FP16. Ускорение инференса в 1,5–2 раза без дополнительной потери точности.

Этап 3: Structured Pruning. Обнуляется 30% весов в Linear-слоях по критерию наименьшей L_2 -нормы. Прунинг применяется к слоям attention projection и feed-forward, нетронутыми остаются слои нормализации и embedding.

Этап 4: INT8 Quantization Analysis. Оценивается применимость INT8-квантизации к каждому слою по метрике чувствительности. Выявлены критические слои (первый и последний Linear в каждом transformer-блоке), для которых квантизация приводит к значительной деградации точности; они исключаются из схемы.

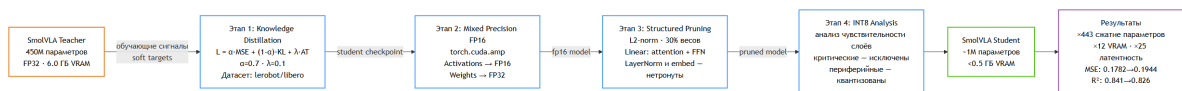


Рис. 4.1. Четырёхэтапный пайплайн оптимизации SmolVLA: дистилляция, FP16, прунинг, INT8-анализ

4.4. Результаты экспериментов

Результаты оптимизации приведены в Табл. 4.2. Совокупный пайплайн обеспечивает сжатие модели в **443 раза** по числу параметров при снижении требований к VRAM в 12 раз.

Таблица 4.2. Результаты оптимизации SmolVLA

Метрика	Teacher	Student	Изменение
Параметры	450M	≈1M	×443 сжатие
VRAM	6,0 ГБ	<0,5 ГБ	×12 сжатие
MSE	0,1782	0,1944	+9,1%
MAE (средн.)	—	—	+5,1%
R^2	0,8412	0,8261	−1,8%
Латентность	450 мс	18 мс	×25 ускорение

Качество сохраняется на уровне выше 90% от исходного по всем трём метрикам. По всем семи степеням свободы манипулятора per-action MAE остаётся в пределах допустимого для лабораторных условий. Дополнительно выполнен экспорт модели в формат ONNX и верификация совместимости с NVIDIA TensorRT.

4.5. Выводы и результаты по главе 4

Экспериментальный модуль SmolVLA Tools демонстрирует принципиальную достижимость бортового VLA-инференса на NVIDIA Jetson Orin Nano при сохранении более 90% точности управления манипулятором и латентности 18 мс — что на два порядка меньше задержки облачного LLM.

Ключевые ограничения: эксперименты проводились исключительно на датасете lerobot/libero (имитация манипулятора), физическая верификация на реальном роботе не выполнялась. Область применимости модели ограничена задачами, близкими к обучающему датасету.

Следующим шагом является интеграция дистиллированной модели как VDA5050 action-handler в Mission Control: при получении VDA5050 order с типом `vla_action` Mission Control запускает локальный инференс SmolVLA вместо передачи команды через ROS 2 навигационный стек. Это добавит уровень индивидуальной бортовой автономности к централизованному контуру AgentsSwarm.

ГЛАВА 5. Верификация системы

5.1. Стратегия тестирования

Стратегия верификации AgentsSwarm охватывает три уровня: *unit-тесты* базовых сервисов и вспомогательных функций; *контрактные тесты* поведения ключевых агентов; *интеграционные тесты* HTTP API. Все тесты реализованы на pytest с использованием Docker Compose для запуска необходимой инфраструктуры (Redis, RabbitMQ, PostgreSQL). Покрытые компоненты: Orchestrator и Interface — два наиболее значимых сервиса системы.

5.2. Тестирование оркестратора

Тестовый набор оркестратора включает 43 теста, организованных по четырём группам.

Unit-тесты базовых сервисов проверяют: корректность health-check эндпоинта, инициализацию SessionManager в Redis, seq-нумерацию StreamCollector при пакетной публикации, поведение retry-механизма при имитации сетевых ошибок.

Контрактные тесты планировщика верифицируют структуру выходных данных MissionPlanner: обязательное наличие полей steps, target_robots и dependencies; корректность ссылок зависимостей; непустоту target_robots для mission-категорий.

Тесты guardrails проверяют граничные сценарии: пустой запрос возвращает standard error-ответ; запрос вне компетенции агента — silent fallback; попытка handoff к несуществующему агенту — перехват на уровне guardrail.

Интеграционные тесты HTTP API проверяют все эндпоинты с реальным FastAPI-приложением. Результат: **43/43 тестов пройдено ✓**. Покрытые сценарии: создание сессии, отправка сообщения, получение статуса задачи, история диалога, принудительная отмена, graceful shutdown.

5.3. Тестирование Interface

Pytest-покрытие Interface включает: эндпоинты управления чатами (создание, получение, удаление), загрузку файлов с верификацией сохранения в MinIO, маршрутизацию к агентам с mock-объектами Celery-задач, авториза-

цию WebSocket-соединений.

Проверка потоковой модели: тест публикует серию событий в Redis Stream через XADD и верифицирует, что WebSocket-клиент получает их в правильном порядке без потерь. Тест завершения сессии проверяет, что сигнал `agent_reply` корректно закрывает XREAD-цикл.

Smoke-тест полного стека запускает `docker-compose` с минимальной конфигурацией (FastAPI + Redis + RabbitMQ + Celery) и выполняет сценарий: `POST /api/chats/ → POST /api/chats/{id}/messages/ → WebSocket /ws/chat/{id} → ожидание события agent_reply`. Тест подтверждает корректную работу всей цепочки без реального LLM-бэкенда.

5.4. Выводы и результаты по главе 5

Автоматизированное тестирование подтвердило корректность работы оркестратора (43/43) и Interface. Ограничения текущего покрытия: отсутствие E2E-тестов с реальным LLM-инференсом (зависимость от GPU-кластера); отсутствие нагрузочного тестирования при параллельных сессиях.

Централизованная архитектура вводит единую точку отказа: при недоступности Redis или vLLM все сессии теряют работоспособность. Формальная верификация LLM-планов — устойчивость к hallucination и нарушениям constraint — остаётся открытым исследовательским вопросом, требующим инструментов в стиле SELP [23].

ЗАКЛЮЧЕНИЕ

В диссертационной работе спроектирована и реализована многоагентная робототехническая платформа AgentsSwarm, обеспечивающая координированное выполнение миссий группой мобильных роботов по пользовательскому запросу на естественном языке.

Основные результаты работы.

Реализованы восемь самостоятельных компонентов: Interface (React + FastAPI + Celery, 168 коммитов, v0.5.0), Orchestrator (FastAPI + OpenAI Agents SDK, 56 коммитов, v0.5.6), vLLM Service (Data Parallel, $2 \times$ Tesla V100, v0.2.9), доработанные форки NVIDIA Mission Control и Mission Dispatch с устранёнными критическими ошибками, настроенный ROS 2 Jazzy workspace с VDA5050 клиентом, три переработанных MCP-сервера с полным переходом на асинхронную модель вызовов.

Пройдено 43 из 43 интеграционных тестов HTTP API оркестратора; записана демонстрация полного воркфлоу от пользовательского запроса до исполнения VDA5050 order роботом Carter в симуляторе Isaac Sim.

Для VLA-модели SmolVLA достигнуто сжатие в 443 раза по числу параметров при сохранении более 90% точности ($MSE +9,1\%$, $R^2 -1,8\%$) и ускорении инференса в 25 раз (450 мс $\rightarrow$ 18 мс, RTX 4090).

Научная и практическая значимость. Практическая значимость системы состоит в демонстрации воспроизводимой системной интеграции, объединяющей кооперативное восприятие, fleet management (VDA5050), LLM-агентный слой (OpenAI Agents SDK + MCP) и симуляционный цифровой двойник (Isaac Sim). Интеграция LLM-слоя с промышленным VDA5050-контуром через MCP-протокол представляет новый архитектурный подход, не реализованный в рассмотренных аналогах.

Перспективы развития:

1. Интеграция дистиллированного SmolVLA как VDA5050 action-handler для бортовой автономности на Jetson Orin Nano.
2. Частичная децентрализация оркестрации для повышения устойчивости к отказам.
3. Формальная верификация LLM-планов через constraint checking и

symbolic validation [23].

4. Перенос системы на физический флот AMR с sim-to-real валидацией.
5. Реализация feature-level cooperative perception между роботами через расширение `ros-mpc`.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

- [1] Хан Ю., Чжан Х., Ли Х. и др. Collaborative Perception in Autonomous Driving: Methods, Datasets and Challenges // IEEE Intelligent Transportation Systems Magazine. 2023. DOI: 10.1109/MITS.2023.3298534. arXiv: 2301.06262.
- [2] Лю С., Гао Ч., Чэнь Ю. и др. Towards Vehicle-to-Everything Autonomous Driving: A Survey on Collaborative Perception // arXiv preprint. 2023. arXiv: 2308.16714.
- [3] Ван Т.-Х., Манивасагам С., Лян М. и др. V2VNet: Vehicle-to-Vehicle Communication for Joint Perception and Prediction // ECCV 2020. DOI: 10.1007/978-3-030-58536-5_36. arXiv: 2008.07519.
- [4] Лю Й.-Ч., Тянь Дж., Глэйзер Н., Кира З. When2com: Multi-Agent Perception via Communication Graph Grouping // CVPR 2020. DOI: 10.1109/CVPR42600.2020.00416. arXiv: 2006.00176.
- [5] Ху Й., Фан С., Лэй З. и др. Where2comm: Communication-Efficient Collaborative Perception via Spatial Confidence Maps // NeurIPS 2022. arXiv: 2209.12836.
- [6] Сюй Р., Сян Х., Ту З. и др. V2X-ViT: Vehicle-to-Everything Cooperative Perception with Vision Transformer // ECCV 2022. DOI: 10.1007/978-3-031-19842-7_7. arXiv: 2203.10638.
- [7] Сюй Р., Сян Х., Ся С. и др. OPV2V: An Open Benchmark Dataset and Fusion Pipeline for Perception with Vehicle-to-Vehicle Communication // ICRA 2022. DOI: 10.1109/ICRA46639.2022.9812038. arXiv: 2109.07644.
- [8] Сюй Р., Ту З., Сян Х. и др. CoBEVT: Cooperative Bird's Eye View Semantic Segmentation with Sparse Transformers // CoRL 2022. arXiv: 2207.02202.
- [9] Чжоу Й. и др. CoPeD — Advancing Multi-Robot Collaborative Perception: A Comprehensive Dataset in Real-World Environments // IEEE Robotics and Automation Letters. 2024. arXiv: 2405.14731.

- [10] Лажуа П.-И., Рамтула Б., У Ф., Бельтраме Дж. Towards Collaborative Simultaneous Localization and Mapping: a Survey of the Current Research Landscape // Field Robotics. 2022. Vol. 2. Pp. 971–1000. DOI: 10.55417/fr.2022032.
- [11] Лажуа П.-И., Бельтраме Дж. Swarm-SLAM: Sparse Decentralized Collaborative Simultaneous Localization and Mapping Framework for Multi-Robot Systems // IEEE Robotics and Automation Letters. 2024. Vol. 9, No. 1. Pp. 475–482. DOI: 10.1109/LRA.2023.3333742.
- [12] VDA, VDMA. VDA 5050: Interface for the Communication Between Automated Guided Vehicles (AGV) and a Master Control. Version 2.0. 2022. URL: <https://github.com/VDA5050/VDA5050> (дата обращения: 01.05.2025).
- [13] ван Дёйкерен Н., Пальмьери Л., Ланге Р., Кляйнер А. An Industrial Perspective on Multi-Agent Decision Making for Interoperable Robot Navigation following the VDA5050 Standard // arXiv preprint. 2023. arXiv: 2311.14615.
- [14] Open Robotics. Open RMF (Open Robotics Middleware Framework). 2021–2024. URL: <https://github.com/open-rmf> (дата обращения: 01.05.2025).
- [15] Иовино М., Скукинс Э., Стируд Й. и др. A Survey of Behavior Trees in Robotics and AI // Robotics and Autonomous Systems. 2022. Vol. 154. P. 104096. DOI: 10.1016/j.robot.2022.104096.
- [16] Чэнь К., Хари К., Хуан Х. и др. FogROS2: An Adaptive Platform for Cloud and Fog Robotics Using ROS 2 // ICRA 2023. arXiv: 2205.09778.
- [17] Сиканд К.С., Зартман Л., Рабии С., Бисвас Дж. Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots // IROS 2021. DOI: 10.1109/IROS51168.2021.9635830.
- [18] Цзэн Ф., Ган В., Хуай З. и др. Large Language Models for Robotics: A Survey // arXiv preprint. 2023. arXiv: 2311.07226.

- [19] Ли П., Ан З., Аббар С., Чжоу Л. Large Language Models for Multi-Robot Systems: A Survey // arXiv preprint. 2025. arXiv: 2502.03814.
- [20] Яо С., Чжао Дж., Юй Д. и др. ReAct: Synergizing Reasoning and Acting in Language Models // ICLR 2023. arXiv: 2210.03629.
- [21] Ан М., Броан А., Браун Н. и др. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances // CoRL 2022. arXiv: 2204.01691.
- [22] Сингх И., Блукис В., Мусавьян А. и др. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models // ICRA 2023. arXiv: 2209.11302.
- [23] Пан Дж., Тан Л. и др. SELP: Generating Safe and Efficient Task Plans for Robot Agents with Large Language Models // ICRA 2025. arXiv: 2409.19471.
- [24] Цзяо Й.-Ч., Линь Й.-П. и др. Swarm-GPT: Combining Large Language Models with Safe Motion Planning for Robot Choreography Design // arXiv preprint. 2023. arXiv: 2312.01059.
- [25] Броан А., Браун Н., Карбахал Дж. и др. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control // CoRL 2023. arXiv: 2307.15818.
- [26] Хуан В., Ван Ч., Чжан Р. и др. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models // CoRL 2023. arXiv: 2307.05973.
- [27] Ким М.Дж., Пертч К., Карамчети С. и др. OpenVLA: An Open-Source Vision-Language-Action Model // CoRL 2024. arXiv: 2406.09246.
- [28] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models // CoRL 2023. arXiv: 2310.08864.
- [29] Блэк К., Браун Н., Дрисс Д. и др. π_0 : A Vision-Language-Action Flow Model for General Robot Control // arXiv preprint. 2024. arXiv: 2410.24164.

- [30] Ретцлафф М. и др. (HuggingFace/LeRobot). SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics // arXiv preprint. 2025. arXiv: 2506.01844.
- [31] Вэнь Дж., Чжу Й., Ли Дж. и др. TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation // IEEE Robotics and Automation Letters. 2025. arXiv: 2409.12514.
- [32] NVIDIA Corporation. Multiple Robot ROS 2 Navigation — NVIDIA Isaac Sim Tutorial. 2024. URL: <https://docs.isaacsim.omniverse.nvidia.com> (дата обращения: 01.05.2025).
- [33] Миттал М., Ю Ч., Ю К. и др. Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments // IEEE Robotics and Automation Letters. 2023. Vol. 8, No. 6. Pp. 3740–3747. DOI: 10.1109/LRA.2023.3270034.
- [34] Жасинто М.Ф., Пинто Дж. и др. Pegasus Simulator: An Isaac Sim Framework for Multiple Aerial Vehicles Simulation // ICUAS 2024. arXiv: 2307.05263.
- [35] Фуллер А., Фан З., Дэй К., Барлоу К. A Survey on AI-Driven Digital Twins in Industry 4.0: Smart Manufacturing and Advanced Robotics // Sensors. 2021. Vol. 21, No. 19. P. 6340. DOI: 10.3390/s21196340.
- [36] Квон В., Ли З., Чжуан С. и др. Efficient Memory Management for Large Language Model Serving with PagedAttention // SOSP 2023. DOI: 10.1145/3600006.3613165.
- [37] Qwen Team, Alibaba Group. Qwen2.5 Technical Report // arXiv preprint. 2024. arXiv: 2412.15115.
- [38] NVIDIA Corporation. Isaac ROS — Accelerated Computing for ROS 2 [Электронный ресурс]. 2024. URL: https://nvidia-isaac-ros.github.io/getting_started/index.html (дата обращения: 01.05.2025).
- [39] NVIDIA Corporation. Isaac ROS Repositories and Packages [Электронный ресурс]. 2024. URL: <https://nvidia-isaac-ros.github.io>

- [io/repositories_and_packages/index.html](https://modelcontextprotocol.io/repositories_and_packages/index.html) (дата обращения: 01.05.2025).
- [40] NVIDIA Corporation. Multiple Robot ROS Navigation (Isaac Sim 4.5) [Электронный ресурс]. 2025. URL: https://docs.isaacsim.omniverse.nvidia.com/4.5.0/ros_tutorials/tutorial_ros_multi_navigation.html (дата обращения: 01.05.2025).
- [41] NVIDIA Omniverse. IsaacSim-ros_workspaces: Official ROS 2 Workspaces for Isaac Sim [Электронный ресурс]. URL: https://github.com/NVIDIA-Omniverse/IsaacSim-ros_workspaces (дата обращения: 01.05.2025).
- [42] Open Robotics. ROS 2 Jazzy Jalisco [Электронный ресурс]. 2024. URL: <https://github.com/ros2> (дата обращения: 01.05.2025).
- [43] Robot Web Tools. rosbridge_suite: WebSocket Bridge for ROS [Электронный ресурс]. URL: https://github.com/RobotWebTools/rosbridge_suite (дата обращения: 01.05.2025).
- [44] NVIDIA Isaac. Mission Control: Fleet Management for VDA5050 Robots [Электронный ресурс]. URL: <https://github.com/nvidia-isaac/mission-control> (дата обращения: 01.05.2025).
- [45] NVIDIA Isaac. Mission Dispatch: VDA5050 Fleet Dispatcher [Электронный ресурс]. URL: <https://github.com/nvidia-isaac/mission-dispatch> (дата обращения: 01.05.2025).
- [46] NVIDIA Corporation. cuOpt: GPU-Accelerated Route Optimization [Электронный ресурс]. 2024. URL: <https://docs.nvidia.com/cuopt/> (дата обращения: 01.05.2025).
- [47] OpenAI. OpenAI Agents SDK [Электронный ресурс]. 2025. URL: <https://github.com/openai/openai-agents-python> (дата обращения: 01.05.2025).
- [48] Anthropic. Model Context Protocol (MCP) [Электронный ресурс]. 2024. URL: <https://modelcontextprotocol.io/> (дата обращения: 01.05.2025).

- [49] RobotMCP. ros-mcp-server: ROS MCP Server [Электронный ресурс]. URL: <https://github.com/robotmcp/ros-mcp-server> (дата обращения: 01.05.2025).
- [50] Lowin J. FastMCP: The Fast, Pythonic Way to Build MCP Servers [Электронный ресурс]. URL: <https://github.com/jlowin/fastmcp> (дата обращения: 01.05.2025).
- [51] vLLM Team. vLLM: Easy, Fast, and Cheap LLM Serving [Электронный ресурс]. URL: <https://github.com/vllm-project/vllm> (дата обращения: 01.05.2025).
- [52] Alibaba Cloud. Qwen2.5-7B-Instruct [Электронный ресурс]. 2024. URL: <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct> (дата обращения: 01.05.2025).
- [53] Ramírez S. FastAPI: Fast Web Framework for Building APIs with Python [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 01.05.2025).
- [54] Redis Ltd. Redis-py: Python Client for Redis [Электронный ресурс]. URL: <https://redis.readthedocs.io/en/stable/> (дата обращения: 01.05.2025).
- [55] Broadcom Inc. RabbitMQ: Open Source Message Broker [Электронный ресурс]. URL: <https://www.rabbitmq.com/> (дата обращения: 01.05.2025).
- [56] Celery Project. Celery: Distributed Task Queue [Электронный ресурс]. URL: <https://docs.celeryq.dev/> (дата обращения: 01.05.2025).
- [57] SQLAlchemy Authors. SQLAlchemy: The Database Toolkit for Python [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/> (дата обращения: 01.05.2025).
- [58] MinIO Inc. MinIO Python SDK [Электронный ресурс]. URL: <https://min.io/docs/minio/linux/developers/python/API.html> (дата обращения: 01.05.2025).

- [59] SQLAlchemy Authors. Alembic: Database Migrations for SQLAlchemy [Электронный ресурс]. URL: <https://alembic.sqlalchemy.org/> (дата обращения: 01.05.2025).
- [60] Meta Platforms Inc. React: The Library for Web and Native User Interfaces [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 01.05.2025).
- [61] Chakra UI Team. Chakra UI: Simple, Modular and Accessible Component Library [Электронный ресурс]. URL: <https://v2.chakra-ui.com/> (дата обращения: 01.05.2025).
- [62] HuggingFace. SmolVLA Base: Vision-Language-Action Model [Электронный ресурс]. 2025. URL: https://huggingface.co/lerobot/smolvla_base (дата обращения: 01.05.2025).
- [63] HuggingFace. LeRobot: Making AI for Robotics More Accessible [Электронный ресурс]. URL: <https://github.com/huggingface/lerobot> (дата обращения: 01.05.2025).
- [64] HuggingFace. LeRobot Documentation [Электронный ресурс]. URL: <https://huggingface.co/docs/lerobot> (дата обращения: 01.05.2025).

ГЛАВА А. Глоссарий предметной области

Термин	Определение
AGV / AMR	Automated Guided Vehicle / Autonomous Mobile Robot — категории мобильных промышленных роботов
Behavior Tree (BT)	Дерево поведения — иерархическая модульная структура управления логикой задач робота
C-SLAM	Collaborative Simultaneous Localization and Mapping — совместная локализация и картографирование несколькими роботами
Cooperative Perception	Кооперативное восприятие — обмен перцептивными данными/признаками между агентами для улучшения наблюдения сцены
Fleet Management	Управление флотом — централизованная диспетчеризация группы роботов
Foundation Model	Базовая модель — крупная модель, обученная на широком корпусе данных и адаптируемая для множества задач
LLM	Large Language Model — большая языковая модель
MCP	Model Context Protocol — открытый протокол взаимодействия LLM-агентов с внешними инструментами и данными
Mission	Миссия — структурированное задание для робота или группы роботов
Occupancy Map	Карта занятости — сетка, описывающая проходимость пространства
Orchestrator	Оркестратор — центральный сервис, координирующий агентов и исполнение задач

Термин	Определение
ROS 2	Robot Operating System 2 — фреймворк для разработки роботизированных систем
Sim-to-Real	Перенос модели или политики из симуляции на физический робот
VDA5050	Открытый стандарт коммуникации между AGV/AMR и fleet manager
VLA	Vision-Language-Action model — мультимодальная модель, генерирующая действия робота
vLLM	Библиотека для эффективного инференса LLM с алгоритмом PagedAttention
Waypoint Graph	Граф путевых точек — граф навигационных узлов на карте
WebSocket	Протокол двунаправленной связи для потоковой передачи данных в реальном времени

ГЛАВА Б. Список сокращений

Сокращение	Расшифровка
AGV	Automated Guided Vehicle
AMR	Autonomous Mobile Robot
API	Application Programming Interface
BT	Behavior Tree
C-SLAM	Collaborative Simultaneous Localization and Mapping
DDS	Data Distribution Service
GPU	Graphics Processing Unit
LLM	Large Language Model
MAE	Mean Absolute Error
MCP	Model Context Protocol
MSE	Mean Squared Error
MQTT	Message Queuing Telemetry Transport
ROS	Robot Operating System
SDK	Software Development Kit
SSE	Server-Sent Events
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
VDA	Verband der Automobilindustrie
VLA	Vision-Language-Action
VRAM	Video Random Access Memory

ГЛАВА В. Технические характеристики инфраструктуры

Вычислительная инфраструктура AgentsSwarm

Машина	Провайдер	Конфигурация	Назначение
Симуляция	Selectel	4 vCPU, 16 ГБ RAM, RTX 4090 (24 ГБ VRAM), 128 ГБ SSD, Ubuntu 24.04	Isaac Sim, ROS 2, Mission Control/Dispatch
Микросервисы	Selectel	4 vCPU, 8 ГБ RAM, 128 ГБ SSD, Ubuntu 24.04	Interface, Orchestrator, Redis, RabbitMQ, MinIO, PostgreSQL
LLM-кластер	MTS Cloud	2 ноды × Tesla V100-PCIE 32 ГБ VRAM, 1,6 ТБ/нода, Ubuntu 24.04	vLLM Data Parallel (Qwen2.5-Instruct)

ГЛАВА Г. Ключевые фрагменты кода

Г.1. Конфигурация динамического выбора транспорта MCP

Листинг Г.1: Выбор транспорта MCP через переменную окружения

```
1 import os
2 from mcp.server import Server
3 from mcp.server.stdio import stdio_server
4 from mcp.server.sse import SseServerTransport
5
6 async def main():
7     transport = os.getenv("MCP_TRANSPORT", "stdio")
8     server = Server("mission-control-mcp")
9     # ... регистрация инструментов ...
10    if transport == "sse":
11        port = int(os.getenv("MCP_PORT", "8080"))
12        async with SseServerTransport(port) as sse:
13            await server.run(sse.read_stream, sse.write_stream)
14    else:
15        async with stdio_server() as (r, w):
16            await server.run(r, w)
```

Г.2. StreamCollector: seq-нумерация событий

Листинг Г.2: StreamCollector: буферизация и seq-нумерация

```
1 class StreamCollector:
2     def __init__(self, task_id: str, redis: Redis):
3         self.task_id = task_id
4         self.redis = redis
5         self._seq = 0
6
7     async def publish(self, event_type: str, data: dict):
8         self._seq += 1
9         payload = {"seq": self._seq, "type": event_type, **data}
10        stream_key = f"task:{self.task_id}:stream"
11        await self.redis.xadd(stream_key, {"payload": json.dumps(payload)})
```

Г.3. Функция потерь Knowledge Distillation

Листинг Г.3: Функция потерь KD для дистилляции SmolVLA

```
1 def distillation_loss(student_out, teacher_out, student_attn,
2                       teacher_attn, alpha=0.7, T=3.0, lam=0.1):
3     mse = F.mse_loss(student_out, teacher_out)
4     s_soft = F.log_softmax(student_out / T, dim=-1)
5     t_soft = F.softmax(teacher_out / T, dim=-1)
6     kl = F.kl_div(s_soft, t_soft, reduction='batchmean') * (T ** 2)
7     at = sum(F.mse_loss(s, t) for s, t in zip(student_attn, teacher_attn))
8     return alpha * mse + (1 - alpha) * kl + lam * at
```

ГЛАВА Д. Протоколы экспериментов

Д.1. Протокол тестирования API оркестратора

Среда: Ubuntu 24.04, Python 3.11, pytest 8.x, Redis 7.x (Docker). Конфигурация тестов: переменные окружения `TEST_MODE=true`, `MOCK_LLM=true` (ответы агентов заменяются mock-объектами). Команда запуска: `pytest tests/ -v --tb=short`. Результат: 43 пройдено, 0 провалено, 0 пропущено. Время выполнения: ≈ 45 с.

Д.2. Протокол эксперимента по дистилляции SmoVLA

Среда: Ubuntu 22.04, CUDA 12.1, PyTorch 2.2, RTX 4090 (24 ГБ VRAM). Датасет: `lerobot/libero` (train: 18 000 траекторий, val: 2 000 траекторий). Гиперпараметры: `lr = 1e-4`, `batch_size = 32`, `epochs = 10`, $T = 3,0$, $\alpha = 0,7$, $\lambda = 0,1$, `student_ratio = 0,1`. Время обучения: ≈ 4 ч.

Д.3. Протокол профилирования инференса

Условия измерения: RTX 4090 (24 ГБ VRAM), CUDA 12.1, `batch_size = 1`, 100 прогонов для усреднения. Метрика: среднее время от подачи тензора до получения выходного вектора действий. Teacher FP32: 450 мс. Student FP16: 18 мс. Ускорение: $25\times$.